Compact Lattice-Based Adaptor Signatures Compatible with ML-DSA Verification

Wang Qi
lightsalted@proton.me

Abstract

We construct an adaptor signature for ML-DSA (FIPS 204) in which keys are ML-DSA keys, pre-signatures have the encoding and length of ML-DSA signatures, and adapted signatures are accepted by the unmodified FIPS 204 verification algorithm, with all of ML-DSA's compression in place. Statements are MLWE samples $\mathbf{R} = \mathbf{A}\mathbf{r}_1 + \mathbf{r}_2$ under the signer's matrix. The obstacle is that adding a witness to the response shifts the vector the verifier rounds; we isolate a public condition on this vector under which the shift changes neither the high-order bits nor the hints, and make it part of pre-verification. Every pre-signature that passes pre-verification, including one produced by a malicious signer, is adaptable with every witness of the tight relation.

We prove extractability and unique extractability for statements chosen by the adversary, and adaptor unforgeability for honestly generated statements, in the classical random oracle model, under the definitions of Gerhart et al. (Eurocrypt 2024) adjusted to the knowledge gap of lattice schemes. The reductions do not rewind and rest on MLWE, a digest form of SelfTargetMSIS and the MSIS instance behind the strong unforgeability of ML-DSA (with one extra column for adaptor unforgeability), up to an oracle-simulation term for Fiat–Shamir with aborts that we bound by 2^{-204} for ML-DSA-44 with ideal masks when signing and pre-signing stop after 4096 iterations. We do not claim unlinkability: in our experiment a holder of the witness recognizes about 4% of fresh ML-DSA-44 signatures as fresh. A Rust prototype produced 200 000 adapted ML-DSA-44 signatures, all accepted by three independent verifiers.

1 Introduction

Adaptor signatures [Poe17, AEE+21] add a conditional step to ordinary signing. The signer produces a pre-signature $\widehat{\sigma}$ on a message m with respect to a statement $\mathbf{R}$. A party that knows a witness for $\mathbf{R}$ can adapt $\widehat{\sigma}$ into a signature σ on m, and anyone who sees both $\widehat{\sigma}$ and σ can extract the witness. Atomic swaps, payment channel networks and oracle-based conditional payments use this to move conditional logic off chain: the chain verifies ordinary signatures, and publishing a signature hands the witness to the other party.

The adaptor signatures in use today are built on Schnorr and ECDSA. ML-DSA, standardized in FIPS 204 [NIST24], is the natural replacement for systems that move to post-quantum signatures, and an adaptor signature whose outputs are genuine ML-DSA signatures would let such systems keep their off-chain protocols without changing the verification rule. The obstacle is compression. An ML-DSA signature $(\tilde{c}, \mathbf{z}, \mathbf{h})$ does not contain the commitment. The verifier recomputes $\mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d$, keeps its high-order bits and corrects them with the hint $\mathbf{h}$. Adapting a pre-signature adds the witness to the response $\mathbf{z}$ and so shifts the recomputed vector. If the shift moves a coefficient across a rounding boundary, or flips the sign of its low-order part at a position where the hint is set, the high-order bits change and the signature is rejected. The lattice-based adaptor signature LAS [EEE20] avoids the problem by not compressing.

Contributions.

- An adaptor signature for ML-DSA whose keys, signing and verification are those of FIPS 204 and whose pre-signatures have the encoding and length of signatures. Every pre-signature that passes pre-verification, whoever produced it, is adaptable with every witness of the tight relation (Theorem 1); the tool is a public condition under which UseHint is stable under short shifts (Lemma 1).

- Extractability and unique extractability for statements chosen by the adversary, and adaptor unforgeability for honestly generated statements, in the classical random oracle model under the definitions of [GSST24] with the knowledge gap of lattice schemes (Theorems 2–4). The reductions do not rewind, and the oracle-simulation term is at most 2^{-204} for ML-DSA-44 with ideal masks, 4096-iteration caps on signing and pre-signing, and the query budgets of Proposition 1.
- A quantitative analysis of unlinkability: one-sided tests with and without the witness, with exact advantage and measured rates of 4.1% and 0.5% on fresh ML-DSA-44 signatures, and an adaptation-time filter with a conditional distribution-matching result in an ideal-transcript experiment, at the cost of adaptability (Section 4.7).
- A Rust implementation for ML-DSA-44 whose adapted signatures were checked byte for byte by three independent ML-DSA verifiers, with all experiments reproducible from recorded seeds (Section 5.2).

Construction. The construction keeps the rounding of the public key, the high-order bits and the hints. A pre-signature is a triple $(\tilde{c}, \widehat{\mathbf{z}}, \mathbf{h})$ in the standard signature encoding; adaptation adds the witness to $\widehat{\mathbf{z}}$ and leaves $\tilde{c}$ and $\mathbf{h}$ unchanged, and the result is an ML-DSA signature. A preliminary version of this construction appeared in the author's Master's thesis [Thesis26] (2026); the present paper corrects the witness relation used there and substantially revises the security analysis.

Two choices make this work. The first is the relation. LAS uses statements $\mathbf{Ar}$ for a wide matrix $\mathbf{A}$. In ML-DSA the matrix has at least as many rows as columns, so when its NTT components have full column rank, $\mathbf{Ar}$ determines $\mathbf{r}$ and Gaussian elimination recovers it (Section 3.1). We use statements $\mathbf{R} = \mathbf{Ar}_1 + \mathbf{r}_2$ with short $\mathbf{r}_1, \mathbf{r}_2$, that is, MLWE samples whose secrets are distributed like ML-DSA secret keys, and we take $\mathbf{A}$ from the signer's public key. The second is a public condition C on the vector $\mathbf{v}$ that pre-verification recomputes. It keeps the low-order bits of $\mathbf{v}$ more than η away from the rounding boundaries, and away from zero at hint positions. Under C, adding a vector of norm at most η changes neither the high-order bits nor the direction of any hint correction (Lemma 1). Anyone can check C, so every pre-signature that passes pre-verification, whoever produced it, is adaptable with every witness of the tight relation, and the signer does not have to prove anything. Without C, 0.64% of honestly generated pre-signatures in our experiments pass pre-verification and fail after adaptation.

Security. We use the definitions of Gerhart et al. [GSST24], which allow several pre-signatures on the same message and pre-signing on statements chosen by the adversary, adjusted to the knowledge gap of lattice constructions [EEE20]: honest witnesses lie in a tight relation $\mathcal{R}_\eta$ and extracted witnesses in a wider relation $\mathcal{R}'$. The bounds of $\mathcal{R}'$ are the two components of the bound ζ' of the MSIS instance behind the strong unforgeability of ML-DSA, and both come out of the same computation (Lemma 3).

Extractability (Theorem 2) reduces to the MLWE instance behind the unforgeability of ML-DSA and to a digest form of its SelfTargetMSIS instance (Remark 5 relates the digest form to the usual one), and the reduction does not rewind. The statistical part of the proof is the simulation of the signing and pre-signing oracles without the secret key, which is the step of the Fiat–Shamir-with-aborts analysis where Barbosa et al. [BBD+23] found and repaired a gap in earlier proofs. We redo their hybrid argument for a family of schemes indexed by statements that the adversary chooses adaptively. To make the number of random-oracle points touched by one oracle query independent of these statements, we stop pre-signing and signing after 4096 iterations, a bound that FIPS 204 permits (Section 2.2). The simulation term is then at most 2^{-204} for ML-DSA-44 with up to 2^{128} hash queries and 2^{63} signing and pre-signing queries each (Proposition 1). For uncapped signing a tail term is added (Corollary 1).

Unique extractability (Theorem 4) reduces directly to $\mathsf{MSIS}_{k,\ell,\zeta'}$ and collision resistance, because extraction refuses signatures whose challenge digest differs from that of the pre-signature. Honest adaptation never changes the digest, so this costs nothing. Adaptor unforgeability for honestly generated statements (Theorem 3) follows from extractability by re-randomizing statements, the random self-reducibility used by Dai et al. [DOY22]. The reduction does not guess which statement the forgery belongs to, so the MSIS term carries no factor Q_{R}; the price is one extra column in the MSIS matrix, and the MLWE term keeps the factor $2Q_{\mathsf{R}}$ from two hybrids over the statements.

Unlinkability. Pre-signing applies two rejection tests that ordinary signing does not, and a party that knows the witness can re-run them on a signature. They never fire on an adapted signature, so their advantage in the unlinkability experiment of [GSST24] equals the probability that one of them fires on a fresh signature (Proposition 2), which was 4.1% for ML-DSA-44 in our experiment; the scheme is unlinkable in that sense only if this probability is negligible, and we do not claim the property. Such a test can only show that a signature was not adapted with this particular witness. In an atomic swap the parties know that anyway; the witness in the definition stands for a witness that becomes known later. An observer without the witness has a public one-sided test that fires on about 0.5% of fresh ML-DSA-44 signatures in our experiment (Section 4.7); what such an observer can distinguish beyond this is not bounded by our analysis. We also analyze a second rejection step at adaptation time. For fixed inputs, in the idealized experiment of Proposition 3 with fresh masks and ideal transcripts, successful adaptations are then distributed like successful signatures of a tightened ML-DSA signer; the price is that about 5.7% of honest pre-signatures can no longer be adapted.

Mask derivation. The mask derivation of ML-DSA cannot be reused for pre-signing as it is. In the deterministic mode, two pre-signatures on the same message for different statements reuse masks, and one pair accepted in the same iteration can reveal $\mathbf{s}_1$. Knowing $\mathbf{s}_1$ already allows signing arbitrary messages (Remark 2). Our pre-signing algorithm puts the statement into the derivation.

1.1 Related work

Definitions. Aumayr et al. [AEE+21] gave the first standalone definitions of adaptor signatures, in which the adversary sees one pre-signature on the challenge message. Dai, Okamoto and Yamamoto [DOY22] strengthened these notions, formalized unlinkability, and built an unlinkable adaptor signature from any signature scheme and any strongly random-self-reducible relation, with LWE among the instantiations. Gerhart, Schröder, Soni and Thyagarajan [GSST24] observed that coin mixing, Blind Hub [QPM+23] and oracle payments produce several pre-signatures on the same message, and that a signer may be asked to pre-sign statements outside the language. Their four properties, extractability, unique extractability, unlinkability and pre-verify soundness, are the definitions we work with. Döttling et al. [DTS+26] give a general approach to adaptor signatures and formalize, as strong canonical signing, the requirement that adapted signatures be distributed like fresh ones, which is close to the unlinkability of [GSST24]. Wang et al. [WYX+26] define weak unlinkability, in which the challenger samples the challenge relation, and argue that it suffices against on-chain observers. Constructions for all NP relations either modify the verification algorithm, so that adapted signatures are no longer signatures of the original scheme, or rely on heavier primitives. Ours is specific to ML-DSA and leaves its verification algorithm untouched.

Post-quantum adaptor signatures. Esgin, Ersoy and Erkin [EEE20] proposed LAS and identified the knowledge gap of lattice-based proofs. LAS is built on a simplified Dilithium without compression, with a wide matrix $[\mathbf{I} \mid \mathbf{A}']$. Its example parameters give signatures of about 3210 bytes, and its adapted signatures are signatures of that variant, not of ML-DSA. The pre-signature adaptability of LAS already covers maliciously generated pre-signatures for witnesses of the tight relation, and this is the notion we prove as well. In our setting the difficulty is that compression makes adaptability depend on how the recomputed vector rounds, which is what the condition C controls. Mohammadali and AlTawy [MA26] construct a lattice-based blind adaptor signature, L-BAS, with computational pre-verify soundness (Section 4.6). Tairi, Moreno-Sanchez and Maffei [TMM21] give a post-quantum adaptor signature from isogenies.

LAS+. Wang et al. [WYX+26] revisit LAS under the definitions of [GSST24]. They show that LAS is not unlinkable: fresh and adapted responses put slightly different mass on the boundary of the response box, and a counting test that needs no witness separates them once enough signatures are available. LAS+ restores a weak form of unlinkability, in which the challenge relation is sampled honestly, by adding a rejection step to signing that gives fresh signatures the distribution of adapted ones, and obtains pre-verify soundness by making an argument of knowledge part of the statement and checking it in pre-verification. Its extractability reduction uses the forking lemma, and no implementation or sizes are reported. The base scheme is still the uncompressed variant of LAS, so its signatures are not ML-DSA signatures, and the rejection step changes signing. Neither fits our setting, in which signing and verification must remain those of FIPS 204. Our

extractability reduction avoids rewinding (Section 4.3), and Section 4.7 compares the boundary effect of LAS with the one we measure.

Fiat–Shamir with aborts and ML-DSA. Our extractability proof follows the corrected oracle-simulation argument of Barbosa et al. [BBD+23] for Fiat–Shamir with aborts [Lyu09, KLS18], including their treatment of matrices with a nontrivial kernel. Pellet-Mary and Seck [PS26] study how HighBits and LowBits behave under addition, the question behind our shift lemma. Kwiatkowski [Kwi26] recomputes the expected number of iterations of ML-DSA signing with all rejection conditions included; his value for ML-DSA-44 agrees with our measurement.

Organization. Section 2 recalls ML-DSA and the security definitions. Section 3 gives the construction and proves adaptability. Section 4 contains the security proofs and the analysis of unlinkability. Section 5 reports parameters, the implementation and an application to atomic swaps, and Section 6 states the limitations. Appendix A proves the oracle-simulation bound and Appendix B explains how its numbers are computed.

2 Preliminaries

2.1 Notation

Let $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with $q = 8380417$. For $r \in \mathbb{Z}_q$, $r \bmod^{\pm} \alpha$ denotes the representative in $(-\alpha/2, \alpha/2]$. The norm $\|\cdot\|_\infty$ of a vector of polynomials is the largest absolute value of the centered representatives of its coefficients; where a difference of two responses is formed, it is taken over the integers. S_η is the set of polynomials with coefficients in $[-\eta, \eta]$, $\widetilde{S}_{\gamma_1}$ the set of polynomials with coefficients in $(-\gamma_1, \gamma_1]$, and B_τ the set of polynomials with exactly τ coefficients in $\{\pm 1\}$ and all others 0. For $\nu > 0$ we write $B(\nu) = \{\mathbf{x} : \|\mathbf{x}\|_\infty < \nu\}$, of the appropriate dimension, and $\mathsf{wt}(\mathbf{h})$ for the number of ones in a hint $\mathbf{h}$. Bold upper-case letters denote matrices and the statement $\mathbf{R} \in R_q^k$. Three letters R occur and are always typeset differently: the ring R_q, the statement $\mathbf{R}$ and the relations $\mathcal{R}_\eta, \mathcal{R}'$.

2.2 ML-DSA

The parameters are the dimensions (k, ℓ), the secret bound η, the challenge weight τ, $\beta = \tau\eta$, the masking bound γ_1, the low-order rounding range γ_2, $d = 13$ dropped bits of the public key, and the hint weight bound ω (Table 1). Let $\alpha = 2\gamma_2$. We write HighBits, LowBits, MakeHint and UseHint for the functions of FIPS 204 with parameter α [NIST24, DKL+18]. $\mathsf{MakeHint}(\mathbf{u}, \mathbf{x})$ outputs, coefficient-wise, $[\mathsf{HighBits}(\mathbf{x}) \neq \mathsf{HighBits}(\mathbf{x}+\mathbf{u})]$. $\mathsf{UseHint}(\mathbf{h}, \mathbf{x})$ outputs $\mathsf{HighBits}(\mathbf{x})_i$ where $h_i = 0$, and $\mathsf{HighBits}(\mathbf{x})_i \pm 1 \bmod (q-1)/\alpha$ where $h_i = 1$, with $+$ if $\mathsf{LowBits}(\mathbf{x})_i > 0$ and $-$ otherwise. w1Encode is the byte encoding of $\mathbf{w}_1$ that FIPS 204 hashes; it is injective.

- KeyGen: expand $\mathbf{A} = \mathsf{ExpandA}(\rho) \in R_q^{k\times\ell}$ from a fresh seed ρ, sample $(\mathbf{s}_1, \mathbf{s}_2) \leftarrow S_\eta^\ell \times S_\eta^k$, and write $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2 = \mathbf{t}_1 2^d + \mathbf{t}_0$. Then $vk = (\rho, \mathbf{t}_1)$ and $sk = (\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$ with $tr = \mathrm{SHAKE256}(vk, 64)$.
- Messages: for the pure interface, a message $m = (M, ctx)$ is framed as $M' = \mathtt{0x00}\|\mathrm{len}(ctx)\|ctx\|M$ and hashed to $\mu = H_{\mathsf{m}}(tr\|M')$, where H_{m} is SHAKE256 with 64 bytes of output.
- Sign: repeat: derive a mask $\mathbf{y} \in \widetilde{S}_{\gamma_1}^\ell$, let $\mathbf{w} = \mathbf{A}\mathbf{y}$, $\mathbf{w}_1 = \mathsf{HighBits}(\mathbf{w})$, $\tilde{c} = H_{\mathsf{c}}(\mu\|\mathsf{w1Encode}(\mathbf{w}_1))$, $c = \mathsf{SampleInBall}(\tilde{c})$ and $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$; restart if $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ or $\|\mathsf{LowBits}(\mathbf{w} - c\mathbf{s}_2)\|_\infty \geq \gamma_2 - \beta$; let $\mathbf{h} = \mathsf{MakeHint}(-c\mathbf{t}_0, \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0)$ and restart if $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$ or $\mathsf{wt}(\mathbf{h}) > \omega$. Output $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$.
- Verify: accept if and only if $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, $\mathsf{wt}(\mathbf{h}) \leq \omega$ and $\tilde{c} = H_{\mathsf{c}}(\mu\|\mathsf{w1Encode}(\mathsf{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d)))$, where $c = \mathsf{SampleInBall}(\tilde{c})$.

The digest $\tilde{c}$ has 32, 48 or 64 bytes for ML-DSA-44/65/87. Masks are derived as $\mathbf{y} = \mathsf{ExpandMask}(\rho''_{\mathsf{S}}, \kappa)$ from $\rho''_{\mathsf{S}} = \mathrm{SHAKE256}(K\|rnd\|\mu, 64)$ and a counter κ, where rnd is a fresh 32-byte string (hedged mode) or all zeros (deterministic mode). All values are canonically encoded, and every algorithm rejects an input that does not decode canonically.

FIPS 204 recommends not to bound the signing loop, but permits a bound that is not too small, in which case signing returns an error and nothing else [NIST24, App. C]: the published text gives 814 iterations [NIST24, App. C, Table 3], raised to 821 in NIST's July 2026 errata [NIST26] after the expected repetition counts were corrected [Kwi26]. We write $\mathsf{Sign}_{K_\mathsf{S}}$ for ML-DSA signing that gives up after K_S iterations. Its successful runs are runs of the standard algorithm, and it outputs the same signatures. Our main theorems use $K_\mathsf{S} = 4096$; Corollary 1 covers the uncapped algorithm.

We use three coefficient-wise facts from the Dilithium specification [DKL+18]:

(F1) If $\|\mathbf{u}\|_\infty \le \gamma_2$, then $\mathsf{UseHint}(\mathsf{MakeHint}(\mathbf{u},\mathbf{x}),\mathbf{x}) = \mathsf{HighBits}(\mathbf{x}+\mathbf{u})$.

(F2) If $\|\mathbf{s}\|_\infty \le b$ and $\|\mathsf{LowBits}(\mathbf{x})\|_\infty < \gamma_2 - b$, then $\mathsf{HighBits}(\mathbf{x}+\mathbf{s}) = \mathsf{HighBits}(\mathbf{x})$ and $\mathsf{LowBits}(\mathbf{x}+\mathbf{s}) = \mathsf{LowBits}(\mathbf{x}) + \mathbf{s}$.

(F3) If $\mathsf{UseHint}(\mathbf{h},\mathbf{x}) = \mathbf{w}_1$, then $\|\mathbf{x} - \alpha\mathbf{w}_1\|_\infty \le \alpha + 1$.

2.3 Model and hard problems

As in the security analyses of ML-DSA [DKL+18, KLS18, BBD+23], the matrix $\mathbf{A}$ is uniform (equivalently, $\mathsf{ExpandA}$ is a random oracle), and the message hash H_m and the challenge hash H_c are random oracles. We treat them as independent oracles. In FIPS 204 both are SHAKE256 and their inputs carry no domain-separation prefix, so this is a modelling assumption, the same one made in the analyses we build on. Masks are modelled as fresh uniform samples for every new input and iteration; the actual derivation is accounted for by a separate term Δ_{mask} (Section 4.3). All proofs are in the classical random oracle model.

Definition 1 (MLWE and MSIS)**.** $\mathsf{MLWE}_{k,\ell,\eta}$ asks to distinguish $(\mathbf{A}, \mathbf{A}\mathbf{s}_1+\mathbf{s}_2)$ from $(\mathbf{A},\mathbf{u})$, where $\mathbf{A} \leftarrow R_q^{k\times\ell}$ and $\mathbf{u} \leftarrow R_q^k$ are uniform and $(\mathbf{s}_1,\mathbf{s}_2) \leftarrow S_\eta^\ell \times S_\eta^k$. $\mathsf{MSIS}_{k,m,\xi}$ asks, given a uniform $\mathbf{B} \in R_q^{k\times m}$, for a nonzero $\mathbf{x}$ with $[\mathbf{B} \,|\, \mathbf{I}_k]\,\mathbf{x} = \mathbf{0}$ and $\|\mathbf{x}\|_\infty \le \xi$.

ML-DSA hashes to a digest and expands the digest into a challenge. We state SelfTargetMSIS in this form.

Definition 2 (SelfTargetMSIS, digest form)**.** Let $G : R_q^k \times \{0,1\}^{512} \to \{0,1\}^{8|\tilde{c}|}$ be a random oracle. Given a uniform $\mathbf{B} \in R_q^{k\times(\ell+1)}$ and access to G, $\mathsf{SelfTargetMSIS}_{G,k,\ell+1,\zeta}$ asks for μ and $\mathbf{y} = (\mathbf{e},\mathbf{z},c) \in R_q^k \times R_q^\ell \times R_q$ with $\|\mathbf{y}\|_\infty \le \zeta$ and

$$c = \mathsf{SampleInBall}\big(G([\mathbf{I}_k \,|\, \mathbf{B}]\,\mathbf{y},\ \mu)\big),$$

where $\mathsf{SampleInBall}$ is the public map of FIPS 204.

The usual formulation [DKL+18, KLS18] uses a random oracle with values in B_τ in place of $\mathsf{SampleInBall}\circ G$. Remark 5 relates the two. Let

$$\zeta = \max\{\gamma_1 - \beta,\ 2\gamma_2 + 1 + 2^{d-1}\tau\}, \qquad \zeta' = \max\{2(\gamma_1-\beta),\ 4\gamma_2+2\}.$$

In the random oracle model the unforgeability of ML-DSA reduces to $\mathsf{MLWE}_{k,\ell,\eta}$ and $\mathsf{SelfTargetMSIS}_{k,\ell+1,\zeta}$, and its strong unforgeability in addition to $\mathsf{MSIS}_{k,\ell,\zeta'}$ [DKL+18, Sec. 4.2]. The Dilithium specification [BDK+21, Table 1] gives lattice-attack estimates for the corresponding LWE and SIS problems; we repeat them in Table 2. They are not a separate assessment of the digest form of Definition 2.

2.4 Adaptor signatures and security games

An adaptor signature for ML-DSA consists of $\mathsf{PreSign}(sk,m,\mathbf{R}) \to \widehat{\sigma}$, $\mathsf{PreVerify}(vk,m,\mathbf{R},\widehat{\sigma}) \to \{0,1\}$, $\mathsf{Adapt}(vk,m,\mathbf{R},\widehat{\sigma},w) \to \sigma$ and $\mathsf{Extract}(vk,\widehat{\sigma},\sigma,\mathbf{R}) \to w$, together with an instance generator $\mathsf{RGen}(vk) \to (\mathbf{R},w)$. $\mathsf{PreSign}$, Adapt and $\mathsf{Extract}$ may output $\perp$, and $\mathsf{PreVerify}$ rejects $\perp$.

Relation pairs. Honest witnesses lie in a tight relation $\mathcal{R}_\eta(vk)$, while witnesses extracted from signatures are only guaranteed to lie in a looser relation $\mathcal{R}'(vk) \supseteq \mathcal{R}_\eta(vk)$ [EEE20]. Both depend on the signer's matrix (Section 3.1); in [GSST24] the relation does not depend on the key, but none of their experiments relies on this. We keep the experiments of [GSST24] with one adjustment: in correctness, adaptability and unlinkability, witnesses are taken from $\mathcal{R}_\eta$, and in extractability and unique extractability an extraction succeeds when its output lies in $\mathcal{R}'$. For $\mathcal{R}' = \mathcal{R}_\eta$ these are the original definitions, with unlinkability stated in its single-challenge form.

Definition 3 (Correctness and adaptability)**.** *Correctness*: for honestly generated keys and $(\mathbf{R}, w) \leftarrow \mathsf{RGen}(vk)$, every output $\widehat{\sigma} \neq \bot$ of $\mathsf{PreSign}(sk, m, \mathbf{R})$ passes $\mathsf{PreVerify}$, $\mathsf{Adapt}(vk, m, \mathbf{R}, \widehat{\sigma}, w)$ passes Verify, and $\mathsf{Extract}$ returns w. *Pre-signature adaptability*: for every $(\mathbf{R}, w) \in \mathcal{R}_\eta(vk)$ and every $\widehat{\sigma}$ with $\mathsf{PreVerify}(vk, m, \mathbf{R}, \widehat{\sigma}) = 1$, possibly produced by a malicious signer, $\mathsf{Verify}(vk, m, \mathsf{Adapt}(vk, m, \mathbf{R}, \widehat{\sigma}, w)) = 1$.

Oracles. The challenger generates (vk, sk) and keeps a set S of signed messages and a table P of pre-signatures. The adversary may call:

- $\mathcal{O}_{\mathsf{S}}(m)$: add m to S and return $\mathsf{Sign}_{K_{\mathsf{S}}}(sk, m)$. The message is added even if the output is $\bot$.
- $\mathcal{O}_{\mathsf{P}}(m, \mathbf{R})$: compute $\widehat{\sigma} \leftarrow \mathsf{PreSign}(sk, m, \mathbf{R})$; if $\widehat{\sigma} \neq \bot$, add $(m, \mathbf{R}, \widehat{\sigma})$ to P; return $\widehat{\sigma}$. The adversary does not have to show that $\mathbf{R}$ is in the language.
- $\mathsf{NewR}()$: run $(\mathbf{R}, w) \leftarrow \mathsf{RGen}(vk)$ and return $\mathbf{R}$.

Extractability. The adversary outputs (m^*, σ^*). It wins if $m^* \notin S$, $\mathsf{Verify}(vk, m^*, \sigma^*) = 1$, and $\mathsf{Extract}(vk, \widehat{\sigma}, \sigma^*, \mathbf{R}) \notin \mathcal{R}'(vk)$ for every entry $(m^*, \mathbf{R}, \widehat{\sigma})$ of P. If P has no entry for m^*, the last condition holds trivially.

Unique extractability. The adversary outputs $(m, \mathbf{R}, \widehat{\sigma}, \sigma, \sigma')$ with $\sigma \neq \sigma'$ as byte strings. It wins if $\mathsf{PreVerify}(vk, m, \mathbf{R}, \widehat{\sigma}) = 1$, both signatures verify on m, and both $\mathsf{Extract}(vk, \widehat{\sigma}, \sigma, \mathbf{R})$ and $\mathsf{Extract}(vk, \widehat{\sigma}, \sigma', \mathbf{R})$ lie in $\mathcal{R}'(vk)$. The pre-signature need not come from the oracle.

Adaptor unforgeability for honest statements. The adversary outputs (m^*, σ^*), under the restriction that every query $\mathcal{O}_{\mathsf{P}}(m^*, \mathbf{R})$ uses a statement $\mathbf{R}$ returned by NewR; pre-signing queries on other messages are unrestricted. It wins if $m^* \notin S$ and $\mathsf{Verify}(vk, m^*, \sigma^*) = 1$. This is the notion of [AEE+21] with several pre-signatures on the challenge message.

Unlinkability. We state the single-challenge form: the adversary chooses m and $(\mathbf{R}, w) \in \mathcal{R}_\eta(vk)$ and receives either

$$\mathsf{Adapt}\big(vk, m, \mathbf{R}, \mathsf{PreSign}(sk, m, \mathbf{R}), w\big) \qquad \text{or} \qquad \mathsf{Sign}(sk, m);$$

its advantage in telling the two apart must be negligible. The definition of [GSST24] in addition gives the adversary the signing and pre-signing oracles and lets it ask for several challenges; an adversary against the single-challenge form is an adversary against that definition, which is all the one-sided distinguishing tests of Section 4.7 need. Since our algorithms may return $\bot$, we compare outputs conditioned on success; the tests of Section 4.7 apply either way.

Pre-verify soundness. For statements $\mathbf{R}$ outside the language $L_{\mathcal{R}'}$, no pre-signature passing $\mathsf{PreVerify}$ exists (statistical version), or none can be found efficiently (computational version). We do not claim this property (Section 4.6).

3 Construction

3.1 Relation and public condition

Adaptation adds the witness to the response vector, so the statement has to be defined with respect to the matrix $\mathbf{A} = \mathsf{ExpandA}(\rho)$ that the signer uses. We take $\mathbf{A}$ from the signer's verification key. Key generation is that of ML-DSA; Remark 1 describes a variant in which the matrix is fixed in public parameters.

Why not $\mathbf{R} = \mathbf{Ar}$. LAS uses a wide matrix $[\mathbf{I}_n \mid \mathbf{A}']$, so that a statement $\mathbf{Ar}$ has many preimages and finding a short one is an ISIS problem. All ML-DSA parameter sets have $k \geq \ell$. If the top $\ell \times \ell$ block of $\mathbf{A}$ is invertible in every NTT component, then $\mathbf{Ar}$ determines $\mathbf{r}$, and Gaussian elimination in the NTT domain computes it; our script demonstrates this recovery for ML-DSA-44 parameters. For a square matrix some component is singular with probability about $256/q \approx 2^{-15}$, which does not make the relation hard. When a relation is moved to another scheme's matrix, the condition under which it is hard, here a sufficiently wide matrix, has to be checked again.

Relation pair.

$$\mathcal{R}_\eta(vk) = \big\{(\mathbf{R}, (\mathbf{r}_1, \mathbf{r}_2)) : \mathbf{R} = \mathbf{Ar}_1 + \mathbf{r}_2,\ \|\mathbf{r}_1\|_\infty \leq \eta,\ \|\mathbf{r}_2\|_\infty \leq \eta\big\}, \tag{1}$$

$$\mathcal{R}'(vk) = \big\{(\mathbf{R}, (\mathbf{r}_1, \mathbf{r}_2)) : \mathbf{R} = \mathbf{Ar}_1 + \mathbf{r}_2,\ \|\mathbf{r}_1\|_\infty < 2(\gamma_1 - \beta),\ \|\mathbf{r}_2\|_\infty \leq 4\gamma_2 + 2\big\}. \tag{2}$$

$\mathsf{RGen}(vk)$ samples $(\mathbf{r}_1, \mathbf{r}_2) \leftarrow S_\eta^\ell \times S_\eta^k$ and outputs $\mathbf{R} = \mathbf{Ar}_1 + \mathbf{r}_2$ with witness $(\mathbf{r}_1, \mathbf{r}_2)$. The witness is distributed like an ML-DSA secret key, so under $\mathsf{MLWE}_{k,\ell,\eta}$ the statement is indistinguishable from uniform; this is the assumption that protects the signing key. The two bounds of $\mathcal{R}'$ are the two components of ζ' (Lemma 3).

The public condition. For $\mathbf{v} \in R_q^k$, a hint $\mathbf{h}$ and a margin $b \geq 1$, let $\mathsf{C}_b(\mathbf{v}, \mathbf{h}) = 1$ if and only if

(C1) $\|\mathsf{LowBits}(\mathbf{v})\|_\infty < \gamma_2 - b$, and

(C2) $\mathsf{LowBits}(\mathbf{v})_i \notin (-b, b]$ for every position i with $h_i = 1$.

We write $\mathsf{C} = \mathsf{C}_\eta$; the margin 2η is used in Section 4.7. (C1) keeps the high-order bits fixed under a shift of norm at most b, and (C2) keeps the direction of each hint correction fixed. The interval in (C2) is half-open because $\mathsf{UseHint}$ treats a zero low-order part as non-positive.

Remark 1 (Shared matrix). A deployment may instead fix ρ in public parameters, so that all keys share $\mathbf{A}$. Then RGen does not depend on the key, a single statement can be used with every signer, and atomic swaps become simpler (Section 5.3). The cost is that keys no longer have their own matrix, so precomputation against $\mathbf{A}$ is shared by all users. The algorithms and proofs below are unchanged.

3.2 Algorithms

Key generation, signing and verification are those of ML-DSA, with signing capped at K_S iterations in the security theorems. The adaptor algorithms are Algorithms 1–3. A statement is encoded as $\mathsf{enc}(\mathbf{R})$, which writes each coefficient in $[0, q-1]$ as a 23-bit little-endian string in a fixed order; decoding rejects coefficients $\geq q$. Pre-signing derives its masks from

$$\rho''_\mathsf{P} = \mathrm{SHAKE256}(K \,\|\, rnd \,\|\, \mu \,\|\, \mathsf{enc}(\mathbf{R}),\, 64),$$

while ordinary signing keeps $\rho''_\mathsf{S} = \mathrm{SHAKE256}(K\|rnd\|\mu, 64)$. The fields K, rnd and μ have fixed lengths and $\mathsf{enc}(\mathbf{R})$ has a fixed nonzero length, so the inputs of the two derivations never coincide. Compared with ML-DSA signing, $\mathsf{PreSign}$ differs in the mask seed, in the commitment, which includes $\mathbf{R}$, in the response bound, which is tightened by η, and in the additional check of C.

The vector $\mathbf{v}$ computed in $\mathsf{PreVerify}$ equals the one in $\mathsf{PreSign}$:

$$\mathbf{A}\widehat{\mathbf{z}} - c\mathbf{t}_1 2^d + \mathbf{R} = \mathbf{Ay} + c(\mathbf{As}_1 - \mathbf{t}) + c\mathbf{t}_0 + \mathbf{R} = \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0.$$

Algorithm 1 $\mathsf{PreSign}(sk, m, \mathbf{R})$

if m or $\mathbf{R}$ is not validly encoded **return** $\perp$
$\mu := H_{\mathsf{m}}(tr \,\|\, M')$; $\quad \rho''_{\mathsf{P}} := \mathrm{SHAKE256}(K \| rnd \| \mu \| \mathsf{enc}(\mathbf{R}), 64)$; $\quad \kappa := 0$
for $a = 1, \ldots, K_{\mathsf{P}}$ **do**
  $\mathbf{y} := \mathsf{ExpandMask}(\rho''_{\mathsf{P}}, \kappa)$; $\quad \kappa := \kappa + \ell$
  $\mathbf{w} := \mathbf{A}\mathbf{y} + \mathbf{R}$; $\quad \mathbf{w}_1 := \mathsf{HighBits}(\mathbf{w})$
  $\tilde{c} := H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1))$; $\quad c := \mathsf{SampleInBall}(\tilde{c})$; $\quad \widehat{\mathbf{z}} := \mathbf{y} + c\mathbf{s}_1$
  if $\|\widehat{\mathbf{z}}\|_\infty \geq \gamma_1 - \beta - \eta$ **or** $\|\mathsf{LowBits}(\mathbf{w} - c\mathbf{s}_2)\|_\infty \geq \gamma_2 - \beta$ **then continue**
  $\mathbf{v} := \mathbf{w} - c\mathbf{s}_2 + c\mathbf{t}_0$; $\quad \mathbf{h} := \mathsf{MakeHint}(-c\mathbf{t}_0, \mathbf{v})$
  if $\|c\mathbf{t}_0\|_\infty \geq \gamma_2$ **or** $\mathsf{wt}(\mathbf{h}) > \omega$ **or** $\mathsf{C}_\eta(\mathbf{v}, \mathbf{h}) = 0$ **then continue**
  return $\widehat{\sigma} := (\tilde{c}, \widehat{\mathbf{z}}, \mathbf{h})$
return $\perp$

Algorithm 2 $\mathsf{PreVerify}(vk, m, \mathbf{R}, \widehat{\sigma})$

if $\widehat{\sigma} = \perp$ or an input does not decode canonically **return** 0
parse $\widehat{\sigma} = (\tilde{c}, \widehat{\mathbf{z}}, \mathbf{h})$; $\quad \mu := H_{\mathsf{m}}(\mathrm{SHAKE256}(vk, 64) \,\|\, M')$; $\quad c := \mathsf{SampleInBall}(\tilde{c})$
$\mathbf{v} := \mathbf{A}\widehat{\mathbf{z}} - c\mathbf{t}_1 2^d + \mathbf{R}$; $\quad \mathbf{w}'_1 := \mathsf{UseHint}(\mathbf{h}, \mathbf{v})$
return $[\|\widehat{\mathbf{z}}\|_\infty < \gamma_1 - \beta - \eta] \wedge [\mathsf{wt}(\mathbf{h}) \leq \omega] \wedge [\mathsf{C}_\eta(\mathbf{v}, \mathbf{h}) = 1] \wedge [\tilde{c} = H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}'_1))]$

Hence anyone can check C.

Adapt checks only the size of $\mathbf{r}_1$, which is what it adds to the signature; correctness assumes that the caller supplies a witness in $\mathcal{R}_\eta$. $\mathsf{Extract}$ does not run Verify; the games and protocols that call it check the signature themselves.

Remark 2 (Mask derivation). Suppose $\mathsf{PreSign}$ derived its masks from (K, rnd, μ, κ) only, as ML-DSA signing does, and consider the deterministic mode, where rnd is fixed. Two pre-signing runs on the same message with statements $\mathbf{R} \neq \mathbf{R}'$ then use the same $\mathbf{y}$ in each iteration, while their commitments $\mathsf{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{R})$ and $\mathsf{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{R}')$, and hence their challenges c and c', differ. If both runs accept in the same iteration, $\widehat{\mathbf{z}} - \widehat{\mathbf{z}}' = (c - c')\mathbf{s}_1$, which gives $\mathbf{s}_1$ when $c - c'$ is invertible in R_q (otherwise a second pair is used). In a Rust diagnostic with this naive derivation (20 000 pairs, seed 20261008, separate from the runs of Section 5.2), 4980 pairs (24.9%) had $\|\widehat{\mathbf{z}} - \widehat{\mathbf{z}}'\|_\infty \leq 2\beta$, and in all of them $\widehat{\mathbf{z}} - \widehat{\mathbf{z}}' = (c - c')\mathbf{s}_1$ was confirmed against the known $\mathbf{s}_1$; we did not implement the recovery itself.

The vector $\mathbf{s}_1$ alone suffices to sign any message. From the public key one computes $\mathbf{e} := \mathbf{t}_1 2^d - \mathbf{A}\mathbf{s}_1 = \mathbf{s}_2 - \mathbf{t}_0$. To sign, pick $\mathbf{y}$, set $\mathbf{w}_1 := \mathsf{HighBits}(\mathbf{A}\mathbf{y})$, $c := \mathsf{SampleInBall}(H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1)))$ and $\mathbf{z} := \mathbf{y} + c\mathbf{s}_1$, so that $\mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d = \mathbf{A}\mathbf{y} - c\mathbf{e}$. With $\mathbf{h} := \mathsf{MakeHint}(c\mathbf{e}, \mathbf{A}\mathbf{y} - c\mathbf{e})$, fact (F1) gives $\mathsf{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d) = \mathsf{HighBits}(\mathbf{A}\mathbf{y}) = \mathbf{w}_1$ whenever $\|c\mathbf{e}\|_\infty \leq \gamma_2$. Repeating until $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$, $\|c\mathbf{e}\|_\infty \leq \gamma_2$ and $\mathsf{wt}(\mathbf{h}) \leq \omega$ produces valid signatures. Since $\|c\mathbf{e}\|_\infty \leq \beta + \|c\mathbf{t}_0\|_\infty$, the second condition rarely fails, and the other two are of the same kind as the checks of honest signing. Bruinderink and Pessl made a related observation in the context of fault attacks [BP18].

Several pre-signatures on the same message are allowed by the definitions of [GSST24] and occur in applications, so Algorithm 1 puts $\mathsf{enc}(\mathbf{R})$ into the mask seed. Distinct statements, and ordinary signing, then use masks derived from distinct inputs, and in the deterministic mode two runs on the same $(m, \mathbf{R})$ return the same pre-signature. The proofs treat masks as fresh; the cost of the actual derivation is the term Δ_{mask} of Section 4.3.

Remark 3 (Iteration caps). $\mathsf{PreSign}$ returns $\perp$ after K_{P} iterations, and in the security theorems signing returns $\perp$ after K_{S} iterations; we take $K_{\mathsf{P}} = K_{\mathsf{S}} = 4096$. If every iteration accepts independently with probability a, a query fails with probability $(1 - a)^K$. For honest ML-DSA-44 inputs we measured $a \approx 0.22$ for pre-signing (Section 5.2), which gives about 2^{-1460}; this estimate is not used in any proof. The caps exist for the proofs: they bound the number of random-oracle points that one oracle query can touch, for every statement the adversary may choose, so the proofs need no lower bound on the acceptance rate for adversarial statements.

Algorithm 3 $\mathsf{Adapt}(vk, m, \mathbf{R}, \widehat{\sigma}, (\mathbf{r}_1, \mathbf{r}_2))$ and $\mathsf{Extract}(vk, \widehat{\sigma}, \sigma, \mathbf{R})$

Adapt: **if** $\mathsf{PreVerify}(vk, m, \mathbf{R}, \widehat{\sigma}) = 0$ **or** $\|\mathbf{r}_1\|_\infty > \eta$ **return** $\perp$
  return $\sigma := (\tilde{c},\ \widehat{\mathbf{z}} + \mathbf{r}_1,\ \mathbf{h})$ ▷ $\tilde{c}$ and $\mathbf{h}$ are kept
Extract: **if** an input is $\perp$ or does not decode canonically **return** $\perp$
  parse $\widehat{\sigma} = (\hat{\tilde{c}}, \widehat{\mathbf{z}}, \widehat{\mathbf{h}})$ and $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$; **if** $\tilde{c} \neq \hat{\tilde{c}}$ **return** $\perp$ ▷ Remark 6
  $\mathbf{r}_1 := \mathbf{z} - \widehat{\mathbf{z}}$ (over the integers); $\mathbf{r}_2 := \mathbf{R} - \mathbf{A}\mathbf{r}_1$
  return $(\mathbf{r}_1, \mathbf{r}_2)$ if $(\mathbf{R}, (\mathbf{r}_1, \mathbf{r}_2)) \in \mathcal{R}'(vk)$, and $\perp$ otherwise

3.3 Correctness and adaptability

Lemma 1 (Shift invariance of $\mathsf{UseHint}$)**.** *If* $\mathsf{C}_b(\mathbf{v}, \mathbf{h}) = 1$ *and* $\|\mathbf{s}\|_\infty \leq b$*, then* $\mathsf{UseHint}(\mathbf{h}, \mathbf{v}+\mathbf{s}) = \mathsf{UseHint}(\mathbf{h}, \mathbf{v})$.

Proof. By (C1) and (F2), $\mathsf{HighBits}(\mathbf{v} + \mathbf{s}) = \mathsf{HighBits}(\mathbf{v})$ and $\mathsf{LowBits}(\mathbf{v} + \mathbf{s}) = \mathsf{LowBits}(\mathbf{v}) + \mathbf{s}$. Where $h_i = 0$, $\mathsf{UseHint}$ outputs the high-order bits, which are unchanged. Where $h_i = 1$, it adds or subtracts one according to the sign of the low-order part. By (C2), $\mathsf{LowBits}(\mathbf{v})_i > b$ implies $\mathsf{LowBits}(\mathbf{v})_i + s_i > 0$, and $\mathsf{LowBits}(\mathbf{v})_i \leq -b$ implies $\mathsf{LowBits}(\mathbf{v})_i + s_i \leq 0$, so the correction keeps its direction.

It remains to check (F2) at the special bucket of $\mathsf{Decompose}$. If $r^+ - r_0 = q - 1$, FIPS 204 sets the high-order part to 0 and decrements the low-order part. These are exactly the centered representatives $r \in [-\gamma_2, -1]$, for which $\mathsf{Decompose}$ returns $(0, r)$. Together with $r \in [0, \gamma_2]$, the bucket with high-order part 0 is the interval $[-\gamma_2, \gamma_2]$ of $\alpha + 1$ values, on which $\mathsf{LowBits}$ is the identity; every other bucket j is $(j\alpha - \gamma_2,\ j\alpha + \gamma_2]$, on which $\mathsf{LowBits}(r) = r - j\alpha$. Under (C1) a shift of norm at most b stays inside its bucket in both cases, so (F2) holds there as well. □

Since the lemma is a statement about single coefficients, we also checked it exhaustively for ML-DSA-44/65/87: for all $v \in \mathbb{Z}_q$, $h \in \{0, 1\}$ and $|s| \leq \eta$ satisfying C_η ($8.38 \cdot 10^7$, $1.51 \cdot 10^8$ and $8.38 \cdot 10^7$ combinations) there is no counterexample.

Lemma 2 (Correctness of pre-signatures)**.** *Every output* $\widehat{\sigma} \neq \perp$ *of* $\mathsf{PreSign}$ *passes* $\mathsf{PreVerify}$*, and* $\mathsf{UseHint}(\mathbf{h}, \mathbf{v}) = \mathbf{w}_1$.

Proof. By (F1) with $\mathbf{u} = -c\mathbf{t}_0$ and $\|c\mathbf{t}_0\|_\infty < \gamma_2$, $\mathsf{UseHint}(\mathbf{h}, \mathbf{v}) = \mathsf{HighBits}(\mathbf{w} - c\mathbf{s}_2)$. By (F2) with $\mathbf{s} = c\mathbf{s}_2$, $b = \beta$ and the low-order check, this equals $\mathsf{HighBits}(\mathbf{w}) = \mathbf{w}_1$. The other checks of $\mathsf{PreVerify}$ were made in $\mathsf{PreSign}$. □

Theorem 1 (Pre-signature adaptability)**.** *For every* $\widehat{\sigma}$ *with* $\mathsf{PreVerify}(vk, m, \mathbf{R}, \widehat{\sigma}) = 1$*, not necessarily produced by an honest signer, and every* $(\mathbf{R}, (\mathbf{r}_1, \mathbf{r}_2)) \in \mathcal{R}_\eta(vk)$*, the signature* $\sigma = \mathsf{Adapt}(vk, m, \mathbf{R}, \widehat{\sigma}, (\mathbf{r}_1, \mathbf{r}_2))$ *is accepted by the unmodified ML-DSA verification algorithm, and* $\mathsf{Extract}(vk, \widehat{\sigma}, \sigma, \mathbf{R}) = (\mathbf{r}_1, \mathbf{r}_2)$.

Proof. Norm. $\|\widehat{\mathbf{z}} + \mathbf{r}_1\|_\infty \leq (\gamma_1 - \beta - \eta - 1) + \eta < \gamma_1 - \beta$, and $\mathbf{h}$ is unchanged, so $\mathsf{wt}(\mathbf{h}) \leq \omega$.

Commitment. Since $\mathbf{A}\mathbf{r}_1 = \mathbf{R} - \mathbf{r}_2$, the verifier computes $\mathbf{A}(\widehat{\mathbf{z}} + \mathbf{r}_1) - c\mathbf{t}_1 2^d = \mathbf{v} - \mathbf{r}_2$. By Lemma 1, $\mathsf{UseHint}(\mathbf{h}, \mathbf{v} - \mathbf{r}_2) = \mathsf{UseHint}(\mathbf{h}, \mathbf{v}) = \mathbf{w}_1'$, and pre-verification has already checked $\tilde{c} = H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1'))$.

Extraction. Adaptation keeps the digest, so the digest check passes; $\mathbf{z} - \widehat{\mathbf{z}} = \mathbf{r}_1$, $\mathbf{R} - \mathbf{A}\mathbf{r}_1 = \mathbf{r}_2$, and $\mathcal{R}_\eta(vk) \subseteq \mathcal{R}'(vk)$. □

Remark 4 (The condition C is needed)*.* Without C, a coefficient of $\mathbf{v}$ within η of a rounding boundary, or a low-order part near zero at a hint position, can make $\mathsf{UseHint}(\mathbf{h}, \mathbf{v} - \mathbf{r}_2)$ differ from $\mathsf{UseHint}(\mathbf{h}, \mathbf{v})$: the pre-signature passes pre-verification and the adapted signature is rejected. In our experiments (Section 5.2) this happened for 2564 of 400 000 honestly generated pre-signatures (0.64%) when C was removed, and pre-verification with C rejects all of them. A malicious signer could select such pre-signatures deliberately, so C has to be part of pre-verification; checking it on the signer's side would not protect the adapting party. Gerhart et al. introduced pre-verify soundness against this kind of signer, motivated by Blind Hub [GSST24, QPM+23]; here Theorem 1 excludes it directly, without a zero-knowledge proof.

4 Security

We work in the model of Section 2.3 with the games of Section 2.4. Correctness and adaptability were shown in Lemma 2 and Theorem 1.

4.1 Extraction lemma

Lemma 3 (Extraction)**.** *Let* $\widehat{\sigma} = (\tilde{c}, \widehat{\mathbf{z}}, \widehat{\mathbf{h}})$ *satisfy* $\mathsf{PreVerify}(vk, m, \mathbf{R}, \widehat{\sigma}) = 1$ *and* $\sigma^* = (\tilde{c}, \mathbf{z}^*, \mathbf{h}^*)$ *satisfy* $\mathsf{Verify}(vk, m, \sigma^*) = 1$*, with the same digest* $\tilde{c}$ *and the same recovered high-order bits*

$$\mathbf{w}_1 = \mathsf{UseHint}(\widehat{\mathbf{h}}, \mathbf{A}\widehat{\mathbf{z}} - c\mathbf{t}_1 2^d + \mathbf{R}) = \mathsf{UseHint}(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c\mathbf{t}_1 2^d).$$

Then $\mathsf{Extract}(vk, \widehat{\sigma}, \sigma^*, \mathbf{R}) \in \mathcal{R}'(vk)$*.*

Proof. The digest check passes. For $\mathbf{r}_1 = \mathbf{z}^* - \widehat{\mathbf{z}}$ we have $\|\mathbf{r}_1\|_\infty \le (\gamma_1 - \beta - 1) + (\gamma_1 - \beta - \eta - 1) < 2(\gamma_1 - \beta)$. Moreover

$$\mathbf{r}_2 = \mathbf{R} - \mathbf{A}\mathbf{r}_1 = \big(\mathbf{A}\widehat{\mathbf{z}} - c\mathbf{t}_1 2^d + \mathbf{R}\big) - \big(\mathbf{A}\mathbf{z}^* - c\mathbf{t}_1 2^d\big),$$

and by (F3) both terms in parentheses lie within distance $\alpha + 1$ of $\alpha\mathbf{w}_1$, so $\|\mathbf{r}_2\|_\infty \le 2\alpha + 2 = 4\gamma_2 + 2$. □

The bounds of $\mathcal{R}'$ coincide with ζ' because the strong unforgeability proof of ML-DSA bounds the same two quantities: a difference of two responses, and a difference of two vectors with the same $\mathsf{UseHint}$ output.

4.2 Oracle simulation with capped iterations

For a fixed statement $\mathbf{R}$, $\mathsf{PreSign}$ is the Fiat–Shamir-with-aborts transform of an identification scheme $\mathsf{ID}_{\mathbf{R}}$ with commitment $\mathbf{w}_1 = \mathsf{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{R})$, challenge $c = \mathsf{SampleInBall}(H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1)))$ and response $\widehat{\mathbf{z}} = \mathbf{y} + c\mathbf{s}_1$, or $\perp$ when a check fails; the hint is a public function of an accepted transcript. Ordinary signing is the case $\mathbf{R} = \mathbf{0}$ with the untightened response bound. The extractability proof replaces both oracles by a simulator that does not know $(\mathbf{s}_1, \mathbf{s}_2)$. Barbosa et al. [BBD+23] showed that this step must also account for the random-oracle points of rejected iterations, whose values are correlated with the rejection. We follow their hybrids, and we count iterations with the caps instead of an expected number of repetitions, because the caps give a bound that holds for every statement and every history.

Fix $(\mathbf{A}, \mathbf{t}, \mathbf{t}_0)$. Let $\nu_{\mathsf{S}} = \gamma_1 - \beta$ and $\mathbf{R} = \mathbf{0}$ for signing, and $\nu_{\mathsf{P}} = \gamma_1 - \beta - \eta$ and $\mathbf{R}$ the queried statement for pre-signing. For $j \in \{\mathsf{S}, \mathsf{P}\}$, an iteration of the simulator passes a box coin with probability

$$\pi_j = \frac{(2\nu_j - 1)^{256\ell}}{(2\gamma_1)^{256\ell}} \tag{3}$$

and is otherwise rejected. If the coin passes, the simulator samples a uniform digest $\tilde{c}$ and a uniform $\mathbf{z} \in B(\nu_j)$, computes

$$\mathbf{x} = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{R}, \qquad \mathbf{v} = \mathbf{x} + c\mathbf{t}_0, \qquad \mathbf{w}_1 = \mathsf{HighBits}(\mathbf{x}), \qquad \mathbf{h} = \mathsf{MakeHint}(-c\mathbf{t}_0, \mathbf{v}),$$

and applies the remaining checks: $\|\mathsf{LowBits}(\mathbf{x})\|_\infty < \gamma_2 - \beta$, $\|c\mathbf{t}_0\|_\infty < \gamma_2$, $\mathsf{wt}(\mathbf{h}) \le \omega$, and for pre-signing $\mathsf{C}_\eta(\mathbf{v}, \mathbf{h}) = 1$. If all pass, it programs $H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1)) := \tilde{c}$ and outputs $(\tilde{c}, \mathbf{z}, \mathbf{h})$. After K_j iterations without success it outputs $\perp$ and programs nothing.

Lemma 4 (Ingredients)**.**

(i) Perfect simulation. *If every iteration of* $\mathsf{Sign}_{K_{\mathsf{S}}}$ *and* $\mathsf{PreSign}$ *draws its digest locally and uniformly, and only the accepted iteration programs* H_{c}*, then the simulator reproduces the joint distribution of outputs, programmed points and* $\perp$ *exactly, for every* $\mathbf{R}$*.*

(ii) Commitment entropy before rejection. *Let* $\mathrm{cr}(\mathbf{A})$ *be the sum over the* 256 *NTT components of the corank of the top* $\ell \times \ell$ *block of* $\mathbf{A}$*. For every* $\mathbf{R}$ *and every* $\mathbf{w}_1$*,*

$$\Pr_{\mathbf{y} \leftarrow \tilde{S}^{\ell}_{\gamma_1}} \big[\mathsf{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{R}) = \mathbf{w}_1\big] \le \varepsilon(\mathbf{A}) := q^{\mathrm{cr}(\mathbf{A})} \Big(\frac{\alpha + 1}{2\gamma_1}\Big)^{256\ell}. \tag{4}$$

(iii) Number of iterations. *With Q_{S} signing and Q_{P} pre-signing queries, the total number N of iterations satisfies, in every execution,*

$$N \le L := K_{\mathsf{S}} Q_{\mathsf{S}} + K_{\mathsf{P}} Q_{\mathsf{P}}. \tag{5}$$

Proof. (i) For every c and every $\mathbf{z} \in B(\nu_j)$, the vector $\mathbf{y} = \mathbf{z} - c\mathbf{s}_1$ lies in $\widetilde{S}^{\ell}_{\gamma_1}$ because $\|c\mathbf{s}_1\|_\infty \le \beta$. So $(\mathbf{y}, \tilde{c}) \mapsto (\mathbf{y} + c\mathbf{s}_1, \tilde{c})$ is a bijection from the pairs that pass the box check onto $B(\nu_j) \times \{0,1\}^{8|\tilde{c}|}$, and the box check passes with probability exactly π_j, independently of $\tilde{c}$. The low-order check together with (F2) gives $\mathsf{HighBits}(\mathbf{w}) = \mathsf{HighBits}(\mathbf{w} - c\mathbf{s}_2)$, and $\mathbf{w} - c\mathbf{s}_2 = \mathbf{A}\mathbf{z} - c\mathbf{t} + \mathbf{R} = \mathbf{x}$, so the programmed point is $\mathsf{HighBits}(\mathbf{x})$. All remaining checks and the hint are functions of $(\mathbf{z}, c, \mathbf{R}, \mathbf{t}, \mathbf{t}_0)$. Rejections are matched iteration by iteration, so the caps and $\perp$ are matched as well.

(ii) Restrict to the top ℓ rows. In the NTT domain, $\mathbf{y} \mapsto \mathbf{A}_{\mathrm{top}}\mathbf{y}$ splits into 256 linear maps on $\mathbb{F}_q^{\ell}$, and the i-th has fibres of size q^{cr_i}. Fixing the high-order bits of a coefficient leaves at most $\alpha + 1$ values for it, so at most $q^{\mathrm{cr}(\mathbf{A})}(\alpha+1)^{256\ell}$ of the $(2\gamma_1)^{256\ell}$ masks give a prescribed $\mathbf{w}_1$.

(iii) holds by construction, independently of the key, the statements and the history. □

Let

$$\varepsilon_D = \mathbb{E}_{\mathbf{A}}\big[\varepsilon(\mathbf{A}) \cdot \mathbf{1}[\mathrm{cr}(\mathbf{A}) \le D]\big], \qquad \delta_D = \Pr_{\mathbf{A}}[\mathrm{cr}(\mathbf{A}) > D].$$

Proposition 1 (Oracle simulation)**.** *Let Δ_{cap} be the statistical distance between the adversary's view in the real experiment, with a random oracle H_{c}, the oracle $\mathsf{Sign}_{K_{\mathsf{S}}}$ and the oracle $\mathsf{PreSign}$ on adaptively chosen statements, and its view when both oracles are replaced by the simulator above. Let Q_{H} bound the adversary's queries to H_{c}, with the verification of its final output counted as one more. Then, for every $D \ge 0$,*

$$\Delta_{\mathrm{cap}} \le \big[2(Q_{\mathsf{H}}+1)L + L^2\big]\varepsilon_D + \delta_D. \tag{6}$$

The statements may be chosen adaptively and may lie outside the language; outputs $\perp$ are part of the view.

The proof is in Appendix A. The bound uses neither an expected number of repetitions nor a lower bound on the acceptance rate for adversarial statements. With $Q_{\mathsf{S}}, Q_{\mathsf{P}} \le 2^{63}$ and $K_{\mathsf{S}} = K_{\mathsf{P}} = 2^{12}$ we have $L \le 2^{76}$, and for ML-DSA-44 with $Q_{\mathsf{H}} \le 2^{128}$ and $D = 11$,

$$\log_2 \varepsilon_{11} \approx -409.39, \qquad \log_2 \delta_{11} \approx -209.20, \qquad \Delta_{\mathrm{cap}} \le 2^{-204.34}.$$

For ML-DSA-65 and ML-DSA-87, with $Q_{\mathsf{H}} \le 2^{192}$ and 2^{256}, the bound is below 2^{-846} and 2^{-1258} (Appendix B). These numbers bound the simulation term only; the other terms of Theorem 2 are separate.

4.3 Extractability

Theorem 2 (Extractability)**.** *For every adversary $\mathcal{A}$ in the extractability game, with ideal masks and the caps $K_{\mathsf{S}}, K_{\mathsf{P}}$, there are adversaries $\mathcal{B}$ and $\mathcal{C}$, running in about the time of $\mathcal{A}$ plus L simulated iterations, such that*

$$\mathrm{Adv}^{\mathrm{ext}}_{\mathcal{A}} \le \Delta_{\mathrm{cap}} + \mathrm{Adv}^{\mathsf{MLWE}}_{k,\ell,\eta}(\mathcal{B}) + \mathrm{Adv}^{\mathsf{SelfTargetMSIS}}_{G,k,\ell+1,\zeta}(\mathcal{C}) + \varepsilon_{\mu}, \tag{7}$$

where ε_{μ} is the probability that two distinct framed messages have the same μ.

Proof. The structure is that of the unforgeability proof of ML-DSA, with pre-signatures added to the simulation. ML-DSA cannot be used as a black box, since the reduction has to program the random oracle that the unforgeability game controls; this is the transparent-reduction approach of [GSST24].

Game G_0. The extractability game, in which $\mathbf{t}_0$ is in addition given to the adversary at the start. This only strengthens the adversary.

Game G_1. Both oracles are replaced by the simulator of Section 4.2, which uses $(\mathbf{t}, \mathbf{t}_0)$ and no short secret. By Proposition 1, $|\Pr[G_0] - \Pr[G_1]| \le \Delta_{\mathrm{cap}}$.

Game G_2. $\mathbf{t}$ is replaced by a uniform vector, and $\mathbf{t}_1, \mathbf{t}_0$ are computed from it. The simulator never used $(\mathbf{s}_1, \mathbf{s}_2)$, so $|\Pr[G_1] - \Pr[G_2]| \le \mathrm{Adv}^{\mathsf{MLWE}}_{k,\ell,\eta}(\mathcal{B})$. The simulator still runs at most L iterations.

Now let $(m^*, \sigma^* = (\tilde{c}^*, \mathbf{z}^*, \mathbf{h}^*))$ be a winning output in G_2, and let $\mathbf{w}_1^* = \mathsf{UseHint}(\mathbf{h}^*, \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 2^d)$, so that $\tilde{c}^* = H_{\mathsf{c}}(\mu^* \| \mathsf{w1Encode}(\mathbf{w}_1^*))$. Consider the point $(\mu^*, \mathbf{w}_1^*)$.

(a) If it was programmed by a pre-signing query on $(m^*, \mathbf{R})$, the corresponding entry of P has digest $\tilde{c}^*$ and, by Lemma 2, recovers $\mathbf{w}_1^*$. By Lemma 3, extraction from this entry succeeds, and $\mathcal{A}$ does not win.

(b) If it was programmed by a signing query on some m, or by a pre-signing query on $(m, \mathbf{R})$ with $m \neq m^*$, then two distinct framed messages share μ^* (a signing query on m^* itself is excluded since $m^* \notin S$). This is the event ε_μ.

(c) Otherwise the point was not programmed. Then $\mathcal{C}$ obtains a solution of $\mathsf{SelfTargetMSIS}_{G,k,\ell+1,\zeta}$ as follows.

$\mathcal{C}$ receives $\mathbf{B}$ and sets $\mathbf{B} = [\mathbf{A} \mid -\mathbf{t}]$, which is uniform in G_2. It answers each query $H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1))$ at a point that the simulator has not programmed by $G(\alpha\mathbf{w}_1, \mu)$; the map $\mathbf{w}_1 \mapsto \alpha\mathbf{w}_1$ is injective on the range of $\mathsf{HighBits}$. Programmed points are answered from the simulator's table. In case (c), $\tilde{c}^* = G(\alpha\mathbf{w}_1^*, \mu^*)$. Let $\mathbf{u} = \mathbf{A}\mathbf{z}^* - c^*\mathbf{t}_1 2^d$ and $\mathbf{e} = \alpha\mathbf{w}_1^* - \mathbf{u} + c^*\mathbf{t}_0$. Since $\mathbf{t} = \mathbf{t}_1 2^d + \mathbf{t}_0$,

$$[\mathbf{I}_k \mid \mathbf{A} \mid -\mathbf{t}]\,(\mathbf{e}, \mathbf{z}^*, c^*)^\top = \alpha\mathbf{w}_1^*,$$

where $\|\mathbf{e}\|_\infty \leq \alpha + 1 + 2^{d-1}\tau$ by (F3) and $\|\mathbf{z}^*\|_\infty < \gamma_1 - \beta$. Hence $\|(\mathbf{e}, \mathbf{z}^*, c^*)\|_\infty \leq \zeta$, and $c^* = \mathsf{SampleInBall}(G(\alpha\mathbf{w}_1^*, \mu^*))$. Collisions between programmed points and earlier queries are already accounted for in Δ_{cap}.

In case (a) extraction succeeds deterministically, with no assumption on $\mathbf{R}$. This is the role of the relation pair $(\mathcal{R}_\eta, \mathcal{R}')$. □

The reduction runs $\mathcal{A}$ once and does not rewind; so do those of Theorems 3 and 4. In Theorems 2 and 4 the computational terms carry no factor depending on the number of queries. In Theorem 3, statement re-randomization removes the target-statement guessing loss from the MSIS term, while the MLWE term keeps the factor $2Q_{\mathsf{R}}$.

Remark 5 (The usual form of SelfTargetMSIS). Suppose the XOF inside $\mathsf{SampleInBall}$ is also modelled as a random oracle. A solver for the digest form then yields a solver for the usual form, in which a random oracle H^* with values in B_τ replaces $\mathsf{SampleInBall} \circ G$. The reduction answers each new query x with a fresh digest $\tilde{c}_x$ and programs the XOF at $\tilde{c}_x$ with a stream for which $\mathsf{SampleInBall}$ outputs $H^*(x)$. Such a stream is easy to sample, because the shuffle inside $\mathsf{SampleInBall}$ can be run backwards from its output, and rejected bytes can be filled in at random. The reduction fails only if two digests collide or the adversary queried the XOF at a digest before it was assigned, which happens with probability at most $(Q_{\mathsf{H}} + Q_{\mathsf{X}} + L + 1)^2\, 2^{-8|\tilde{c}|}$, where Q_{X} counts XOF queries. This is the generic collision bound for a digest of $8|\tilde{c}|$ bits. It is small only when the number of queries is well below $2^{4|\tilde{c}|}$, which is why Theorem 2 is stated with the digest form.

Actual masks. An implementation expands masks from a seed and a counter rather than sampling them freshly. Let Δ_{mask} be the advantage in distinguishing, within the query budget of the experiment, the actual mask streams from an idealized oracle that returns fresh uniform masks for every new input and iteration. The input of a mask stream consists of the algorithm (signing or pre-signing), the key, the framed message, *rnd* and, for pre-signing, $\mathsf{enc}(\mathbf{R})$; signing and pre-signing with $\mathbf{R} = \mathbf{0}$ are different inputs. In the deterministic mode a repeated query is answered by replaying the stored output, including $\perp$. With actual masks, Δ_{mask} is added to the right-hand side of (7), and likewise in Theorem 3 and Proposition 3. This term can be bounded under a PRF assumption on the keyed mask derivation; we do not give a concrete value.

Corollary 1 (Uncapped signing)**.** *Keep the cap K_{P} on pre-signing, and let $\delta_{\mathsf{S}}(K_{\mathsf{S}})$ be the probability that some signing query of the experiment needs more than K_{S} iterations. If $\mathsf{Sign}_{K_{\mathsf{S}}}$ is replaced by uncapped ML-DSA signing, Theorem 2 holds with $\delta_{\mathsf{S}}(K_{\mathsf{S}})$ added to the right-hand side of* (7).

Proof. Run both experiments with the same randomness. They coincide until the first signing query that exceeds K_{S} iterations. □

With $K_{\mathsf{S}} = 4096$ the term δ_{S} is negligible for honest keys if each iteration of signing accepts with probability about 1/4.36 [Kwi26], but turning this into a bound that holds uniformly over keys requires an analysis of the full acceptance predicate that we do not give.

4.4 Adaptor unforgeability

Theorem 3 (Adaptor unforgeability for honest statements)**.** *For every adversary $\mathcal{A}$ in the adaptor unforgeability game of Section 2.4, making at most Q_{R} queries to* NewR*, there are adversaries $\mathcal{A}'$, $\mathcal{B}'$ and $\mathcal{D}$, running in about the time of $\mathcal{A}$, such that*

$$\mathrm{Adv}^{\mathrm{aeuf}}_{\mathcal{A}} \le \mathrm{Adv}^{\mathrm{ext}}_{\mathcal{A}'} + 2Q_{\mathsf{R}} \cdot \mathrm{Adv}^{\mathsf{MLWE}}_{k,\ell,\eta}(\mathcal{B}') + \mathrm{Adv}^{\mathsf{MSIS}}_{k,\ell+1,\zeta''}(\mathcal{D}), \qquad \zeta'' = \max\bigl\{2(\gamma_1-\beta)+\eta,\ 4\gamma_2+2+\eta\bigr\},$$

where $\mathrm{Adv}^{\mathrm{ext}}_{\mathcal{A}'}$ *is bounded by Theorem 2.*

Proof. Hybrid H_0 is the real game. In H_1, every statement returned by NewR is a fresh uniform vector; $|\Pr[H_0] - \Pr[H_1]| \le Q_{\mathsf{R}} \cdot \mathrm{Adv}^{\mathsf{MLWE}}_{k,\ell,\eta}$ by a hybrid over the statements, since the distinguisher can generate $(\mathbf{s}_1, \mathbf{s}_2)$ itself for the given $\mathbf{A}$ and run the whole game. In H_2, a uniform $\mathbf{b} \in R_q^k$ is sampled at the start, and the i-th call to NewR samples $\mathbf{u}_i \leftarrow S_\eta^\ell$, $\mathbf{e}_i \leftarrow S_\eta^k$ and returns $\mathbf{R}_i = \mathbf{b} + \mathbf{A}\mathbf{u}_i + \mathbf{e}_i$. Replacing each $\mathbf{A}\mathbf{u}_i + \mathbf{e}_i$ by a uniform vector, one at a time, turns H_2 into H_1, so $|\Pr[H_1] - \Pr[H_2]| \le Q_{\mathsf{R}} \cdot \mathrm{Adv}^{\mathsf{MLWE}}_{k,\ell,\eta}$. This is the random self-reducibility of the relation, as in [DOY22].

In H_2 the statements are generated by public operations from $\mathbf{b}$ and the adversary's view does not depend on their witnesses. Let $\mathcal{A}'$ be the adversary in the extractability game that samples $\mathbf{b}$, runs $\mathcal{A}$, answers NewR as in H_2, and forwards all other queries. Except with probability $\mathrm{Adv}^{\mathrm{ext}}_{\mathcal{A}'}$, a winning output of $\mathcal{A}$ in H_2 comes with an entry $(m^*, \mathbf{R}_i, \widehat{\sigma})$ of P from which extraction succeeds, giving $(\mathbf{r}_1, \mathbf{r}_2) \in \mathcal{R}'(vk)$ with $\mathbf{A}\mathbf{r}_1 + \mathbf{r}_2 = \mathbf{b} + \mathbf{A}\mathbf{u}_i + \mathbf{e}_i$, that is,

$$[\mathbf{A} \,|\, \mathbf{b} \,|\, \mathbf{I}_k] \cdot (\mathbf{r}_1 - \mathbf{u}_i,\ -1,\ \mathbf{r}_2 - \mathbf{e}_i)^\top = \mathbf{0}.$$

The vector is nonzero because of the middle entry, and its norm is at most ζ''.

The MSIS adversary $\mathcal{D}$ receives a uniform $[\mathbf{A} \,|\, \mathbf{b}] \in R_q^{k\times(\ell+1)}$, generates $(\mathbf{s}_1, \mathbf{s}_2)$ for $\mathbf{A}$ itself and plays H_2 with $\mathcal{A}$, answering signing and pre-signing queries honestly. Every statement contains $\mathbf{b}$, so $\mathcal{D}$ succeeds whichever statement the extraction comes from. □

Re-randomization removes the factor Q_{R} that a reduction guessing the target statement would place in front of the MSIS term; the factor $2Q_{\mathsf{R}}$ of the MLWE term remains, since it comes from replacing the statements one at a time in the two hybrids. The price is η in both components of the norm bound. $\mathsf{MSIS}_{k,\ell+1,\zeta''}$ is the instance behind the strong unforgeability of ML-DSA with one more column and a slightly larger bound. It is a different instance, which we have not estimated separately. Since the best known attacks on $\mathsf{MSIS}_{k,m,\zeta'}$ already work in a sublattice spanned by a subset of the columns [DKL+18], we expect the extra column to change the estimate little.

4.5 Unique extractability

Theorem 4 (Unique extractability)**.** *For every adversary $\mathcal{A}$ there are adversaries $\mathcal{B}$ and $\mathcal{D}$ of about the same running time with*

$$\mathrm{Adv}^{\mathrm{uext}}_{\mathcal{A}} \le \mathrm{Adv}^{\mathsf{MSIS}}_{k,\ell,\zeta'}(\mathcal{B}) + \mathrm{Adv}^{\mathrm{coll}}_{H_{\mathsf{c}}}(\mathcal{D}).$$

Proof. $\mathcal{B}$ receives a uniform $\mathbf{A}$, samples $(\mathbf{s}_1, \mathbf{s}_2)$ itself and answers all queries honestly; no simulated oracle is needed. Let $(m, \mathbf{R}, \widehat{\sigma}, \sigma, \sigma')$ be a winning output, with $\widehat{\sigma} = (\hat{\tilde{c}}, \widehat{\mathbf{z}}, \widehat{\mathbf{h}})$, $\sigma = (\tilde{c}, \mathbf{z}, \mathbf{h})$ and $\sigma' = (\tilde{c}', \mathbf{z}', \mathbf{h}')$. Both extractions pass the digest check, so $\tilde{c} = \hat{\tilde{c}} = \tilde{c}'$ and $c = c'$.

Let $\mathbf{w}_1 = \mathsf{UseHint}(\mathbf{h}, \mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d)$ and $\mathbf{w}_1' = \mathsf{UseHint}(\mathbf{h}', \mathbf{A}\mathbf{z}' - c\mathbf{t}_1 2^d)$. Both signatures verify, so $H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1)) = \tilde{c} = H_{\mathsf{c}}(\mu \| \mathsf{w1Encode}(\mathbf{w}_1'))$; if $\mathbf{w}_1 \ne \mathbf{w}_1'$ this is a collision, since w1Encode is injective. Otherwise, by (F3),

$$\|\mathbf{A}(\mathbf{z} - \mathbf{z}')\|_\infty \le \|(\mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d) - \alpha\mathbf{w}_1\|_\infty + \|(\mathbf{A}\mathbf{z}' - c\mathbf{t}_1 2^d) - \alpha\mathbf{w}_1\|_\infty \le 4\gamma_2 + 2.$$

If $\mathbf{z} = \mathbf{z}'$, then $\mathbf{h} \ne \mathbf{h}'$; both hints act on the same vector, and at a position where they differ the outputs of UseHint differ by ± 1 modulo $(q-1)/\alpha > 2$, contradicting $\mathbf{w}_1 = \mathbf{w}_1'$. Hence $\mathbf{z} \ne \mathbf{z}'$, and $(\mathbf{z} - \mathbf{z}', -\mathbf{A}(\mathbf{z} - \mathbf{z}'))$ is a nonzero solution of $\mathsf{MSIS}_{k,\ell,\zeta'}$, with $\|\mathbf{z} - \mathbf{z}'\|_\infty \le 2(\gamma_1 - \beta)$. This is the corresponding step of the strong unforgeability proof of ML-DSA. □

Remark 6 (The digest check). Without the digest check in Extract, two valid signatures with different challenges could both extract into $\mathcal{R}'$ from the same pre-signature, and the proof above would not apply. Refusing to extract across different digests rules this out by definition. It costs honest parties nothing, since adaptation preserves the digest (Theorem 1), and it fits Theorem 2, whose case (a) yields a pre-signature with the digest of the forgery. The check compares digests and not challenges, because SampleInBall need not be injective. Subtracting the two extracted witnesses would also give a short kernel vector of $[\mathbf{A} \mid \mathbf{I}]$, but its second component is only bounded by $8\gamma_2 + 4$, about $q/4$ for ML-DSA-65/87, where the corresponding MSIS instance is not known to be hard; comparing the two signatures avoids this.

The theorem compares two complete signatures. It does not close the knowledge gap: an application that needs a tight witness after extraction still has to obtain one, as in Section 5.3.

4.6 Pre-verify soundness

We do not claim pre-verify soundness. A counting argument, $(4(\gamma_1 - \beta))^{256\ell}(8\gamma_2 + 5)^{256k} \gg q^{256k}$, suggests that almost every $\mathbf{R} \in R_q^k$ has an $\mathcal{R}'$-witness, in which case the property would hold trivially; we have not proved this. Either way, C certifies that the verification computation is stable under small shifts, not that $\mathbf{R} \in L_{\mathcal{R}_\eta}$. In our setting the attack that the property targets, a pre-signature that passes pre-verification but cannot be adapted, is excluded by Theorem 1 as long as a witness in $\mathcal{R}_\eta$ exists. Where the statement comes from a party that may be malicious, applications enforce membership in $L_{\mathcal{R}_\eta}$ with an argument of knowledge (Section 5.3). LAS+ [WYX+26] obtains computational pre-verify soundness by making the argument of knowledge part of the statement, $(\mathbf{R}, \pi)$, and checking π in PreVerify; L-BAS [MA26] proves the same property for its scheme. The same step applies here unchanged: augmenting $\mathcal{R}_\eta$ with an argument of knowledge of a witness and verifying it in PreVerify yields computational pre-verify soundness from the soundness of the argument, with the definition taken over the augmented tight relation. We keep the argument at the protocol level because the swap needs its knowledge extractor and not only its soundness, and because the statement then remains a plain vector $\mathbf{R} \in R_q^k$.

4.7 Unlinkability

Let $\mathbf{u} = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 2^d$ be the vector the verifier computes from a signature $(\tilde{c}, \mathbf{z}, \mathbf{h})$. A party that knows the witness $(\mathbf{r}_1, \mathbf{r}_2)$ can evaluate

$$D_1 = \big[\|\mathbf{z} - \mathbf{r}_1\|_\infty \geq \gamma_1 - \beta - \eta\big], \qquad D_2 = \big[\mathsf{C}_\eta(\mathbf{u} + \mathbf{r}_2, \mathbf{h}) = 0\big].$$

Proposition 2 (One-sided tests with the witness)**.** *On every adapted signature, $D_1 = D_2 = 0$. Consequently, the distinguisher that answers "fresh" exactly when $D_1 \vee D_2 = 1$ has advantage exactly $\Pr_{\text{fresh}}[D_1 \vee D_2 = 1]$ in the unlinkability experiment, and the scheme is unlinkable in the sense of [GSST24] only if this probability is negligible.*

Proof. An adapted signature has $\mathbf{z} - \mathbf{r}_1 = \widehat{\mathbf{z}}$ and $\mathbf{u} + \mathbf{r}_2 = \mathbf{v}$, and pre-verification has checked $\|\widehat{\mathbf{z}}\|_\infty < \gamma_1 - \beta - \eta$ and $\mathsf{C}_\eta(\mathbf{v}, \mathbf{h}) = 1$. Ordinary signing performs neither check. □

If the coefficients of a fresh $\mathbf{z}$ are close to uniform on $B(\gamma_1 - \beta)$ and the low-order parts of $\mathbf{u}$ close to uniform, the two probabilities are approximately

$$\varepsilon_{\mathbf{z}} \approx 1 - \Big(1 - \frac{2\eta}{2(\gamma_1 - \beta) - 1}\Big)^{256\ell}, \qquad \varepsilon_{\mathsf{C}} \approx 1 - \Big(1 - \frac{2\eta + 1}{\alpha}\Big)^{256k},$$

that is 1.55% and 2.65% for ML-DSA-44 (Table 1 gives all three parameter sets). In our Rust experiment the two tests fired on 1.58% and 2.60% of 200 000 fresh ML-DSA-44 signatures, 4.13% combined, and never on an adapted one. A proven lower bound on $\Pr_{\text{fresh}}[D_1 \vee D_2 = 1]$ would need more than this: for a fixed key and challenge the other rejection conditions of signing are functions of $\mathbf{z}$, and how much they thin the outer band of the box is not bounded here (LAS has no such conditions, which is why the bound of [WYX+26] is exact there); the figures above are the heuristic and the measurement. The combined rate is essentially the additional rejection rate of PreSign: the tests re-run the two rejection conditions that pre-signing applies and signing does not.

What the tests show. Both tests are one-sided and need the witness. If one fires, the signature was not adapted with this witness. If neither fires, nothing follows, since about 96% of fresh signatures pass both. In an atomic swap the two parties already know whether a signature was adapted. The definition hands the witness to the adversary to cover witnesses that leak or are published later, and to make the notion composable. The observed distinguishing rate is substantial at the ML-DSA-44 parameters, so we do not claim unlinkability. What is proved is the exact advantage of these two tests; the advantage of other tests, with or without the witness, including tests that use correlations within a signature, the hint, the statement or several signatures, is not bounded by our analysis (Section 6).

Observers without the witness. Without the witness, D_1 and D_2 cannot be evaluated. The vector $\mathbf{u}$, however, is computed from public data, and for every adapted signature (C1) and (F2) give $\|\mathsf{LowBits}(\mathbf{u})\|_\infty \leq \gamma_2 - 1$. Observing a coefficient of $\mathsf{LowBits}(\mathbf{u})$ equal to $\pm\gamma_2$ therefore certifies that a signature was not produced by Adapt. For a fresh signature a coefficient equals $\pm\gamma_2$ with probability about $1/\alpha$, so this is a public one-sided test; under the same uniformity heuristic it fires on about 0.5% of fresh ML-DSA-44 signatures. In a separate run of 20 000 fresh ML-DSA-44 signatures (seed 20261019) it fired on 106, that is 0.53% with a 95% Wilson interval of $[0.44\%, 0.64\%]$, and on none of 20 000 adapted signatures, as the bound above requires. The test of [WYX+26] against LAS is of the same kind: it counts coefficients at the edge of the box and needs no witness. There both fresh and adapted responses reach the boundary and only their mass differs, by about $2.7 \cdot 10^{-6}$ per coefficient at the parameters of LAS, and their Hoeffding bound gives advantage at least 0.9 from about $4 \cdot 10^8$ signatures. Here an adapted signature never reaches $\pm\gamma_2$, and a single fresh ML-DSA-44 signature was recognized with probability about 0.5% in our experiment.

A party that knows $\mathbf{t}_0$, which the security analysis of ML-DSA treats as public, can also evaluate $D_3 = [\|\mathsf{LowBits}(\mathbf{Az} - c\mathbf{t})\|_\infty \geq \gamma_2 - \beta]$. For fresh signatures $D_3 = 0$ always, since this is the low-order check of signing; for adapted signatures the quantity equals $\mathsf{LowBits}(\mathbf{w} - c\mathbf{s}_2) - \mathbf{r}_2$ and can exceed the bound. In our experiment D_3 fired on 0.64% of adapted signatures, identifying them as adapted. Finally, in the deterministic mode a signature can be compared with a deterministic signature on the same message, so the analysis above applies to the hedged mode.

A filter at adaptation time. One can let the adapting party discard adapted signatures that would fail a tightened version of signing. Let Sign' be ML-DSA signing with three conditions tightened to

(T1) $\|\mathbf{z}\|_\infty < \gamma_1 - \beta - 2\eta$,

(T2) $\|\mathsf{LowBits}(\mathbf{Az} - c\mathbf{t})\|_\infty < \gamma_2 - \beta - \eta$ (this is $\|\mathsf{LowBits}(\mathbf{w} - c\mathbf{s}_2)\|_\infty$), and

(T3) $\mathsf{C}_{2\eta}(\mathbf{Az} - c\mathbf{t}_1 2^d, \mathbf{h}) = 1$,

the checks $\|c\mathbf{t}_0\|_\infty < \gamma_2$ and $\mathsf{wt}(\mathbf{h}) \leq \omega$ being unchanged, with the same cap. Signatures of Sign' are ML-DSA signatures. Let $\mathsf{FilterAdapt}$ compute σ as Adapt does and output it only if it satisfies (T1)–(T3). Evaluating (T2) requires $\mathbf{t}_0$, which the signer has to provide once per key. The margin is doubled because filtering with margin η is not enough: with $\widehat{\mathbf{z}}$ uniform on $B(\nu)$, $\nu = \gamma_1 - \beta - \eta$, the adapted $\mathbf{z}$ conditioned on $\mathbf{z} \in B(\nu)$ is uniform on $B(\nu) \cap (B(\nu) + \mathbf{r}_1)$, and the test D_1 still distinguishes.

Proposition 3 (Matching of successful outputs)**.** *Fix an honest key, a message and* $(\mathbf{R}, (\mathbf{r}_1, \mathbf{r}_2)) \in \mathcal{R}_\eta(vk)$. *In the experiment where every iteration uses a fresh mask and a locally sampled digest and only accepted iterations program* H_{c}*, the output of* $\mathsf{FilterAdapt}$ *applied to* $\mathsf{PreSign}$*, conditioned on success, has the same distribution as the output of* Sign' *conditioned on success. This holds with the caps.*

Proof. Let $P(\mathbf{z}, c)$ be the conjunction of (T1)–(T3) and the two unchanged checks, where $\mathbf{h} = \mathsf{MakeHint}(-c\mathbf{t}_0, \mathbf{Az} - c\mathbf{t}_1 2^d)$ is a function of $(\mathbf{z}, c)$. In this experiment Sign' outputs $(\tilde{c}, \mathbf{z}, \mathbf{h})$ with $(\tilde{c}, \mathbf{z})$ uniform on $\{0,1\}^{8|\tilde{c}|} \times B(\gamma_1 - \beta - 2\eta)$ conditioned on $P(\mathbf{z}, c)$.

Let $\mathbf{z} = \widehat{\mathbf{z}} + \mathbf{r}_1$ and $\mathbf{u} = \mathbf{Az} - c\mathbf{t}_1 2^d$, so that $\mathbf{v} = \mathbf{u} + \mathbf{r}_2$ is the vector of $\mathsf{PreSign}$. We show that $P(\mathbf{z}, c)$ implies every check of $\mathsf{PreSign}$ on $\widehat{\mathbf{z}}$ and that $\mathsf{PreSign}$ computes the hint $\mathsf{MakeHint}(-c\mathbf{t}_0, \mathbf{u})$. (T1) gives $\|\widehat{\mathbf{z}}\|_\infty < \gamma_1 - \beta - \eta$. By (T2) and (F2) with $b = \eta$, adding $\mathbf{r}_2$ to $\mathbf{Az} - c\mathbf{t}$ changes neither its high-order bits nor, by more than η, its low-order bits, and $\mathbf{Az} - c\mathbf{t} + \mathbf{r}_2 = \mathbf{A}\widehat{\mathbf{z}} - c\mathbf{t} + \mathbf{R} = \mathbf{w} - c\mathbf{s}_2$ is the vector whose low-order bits $\mathsf{PreSign}$ checks. By (T3) and (F2) with $b = 2\eta$, $\mathsf{HighBits}(\mathbf{v}) = \mathsf{HighBits}(\mathbf{u})$ and $\mathsf{LowBits}(\mathbf{v}) = \mathsf{LowBits}(\mathbf{u}) + \mathbf{r}_2$,

hence $\mathsf{C}_\eta(\mathbf{v}, \mathbf{h}) = 1$. Finally $\mathbf{v} - c\mathbf{t}_0 = (\mathbf{A}\mathbf{z} - c\mathbf{t}) + \mathbf{r}_2$ has the same high-order bits as $\mathbf{u} - c\mathbf{t}_0$, so both algorithms compute the same hint.

So ($\mathsf{PreSign}, \mathsf{FilterAdapt}$) outputs $(\tilde{c}, \widehat{\mathbf{z}} + \mathbf{r}_1, \mathbf{h})$ with $(\tilde{c}, \widehat{\mathbf{z}})$ uniform on $\{0,1\}^{8|\tilde{c}|} \times B(\gamma_1 - \beta - \eta)$ conditioned on $P(\widehat{\mathbf{z}} + \mathbf{r}_1, c)$. Since $B(\gamma_1 - \beta - 2\eta) \subseteq B(\gamma_1 - \beta - \eta) + \mathbf{r}_1$, translation by $\mathbf{r}_1$ maps the preimage bijectively onto $B(\gamma_1 - \beta - 2\eta)$, and $\mathbf{z}$ is uniform there conditioned on $P(\mathbf{z}, c)$, as in Sign'. With the caps, the first accepted iteration of an i.i.d. sequence, conditioned on its existence, has the distribution of one accepted iteration. □

To move to the random oracle model, let δ_F and δ_S be the unconditioned simulation errors of the two sides and $\pi_F, \pi_S > 0$ their success probabilities for the fixed inputs. The conditioned distributions are then within $2\delta_F/\pi_F + 2\delta_S/\pi_S$. Dividing an error averaged over keys by an empirical filter rate would not give a bound for a fixed key.

The filter does not make the scheme unlinkable. In our experiment it rejected 5.75% of honest pre-signatures, so the adaptability of Theorem 1 is lost, and the unlinkability experiment of [GSST24] admits no $\perp$. A malicious signer can choose a pre-signature with a coefficient $\hat{z}_i = \pm(\gamma_1 - \beta - \eta - 1)$, which occurs in about 0.8% of honest pre-signatures; (T1) then holds after adaptation only if $r_{1,i} = \mp\eta$, so whether the adapting party proceeds reveals information about its witness. A retry needs a fresh pre-signature, which in the deterministic mode does not exist. In an atomic swap the signer learns the witness once the swap completes, but an aborted swap may have leaked some coefficients of $\mathbf{r}_1$, so witnesses must not be reused; where the witness has to stay hidden from the signer, as in Blind Hub [QPM+23], the filter should not be used. Sign' needed 10.8% more iterations than ML-DSA signing in our experiment. LAS [EEE20] does not claim unlinkability either; unlinkability together with adaptability in the ML-DSA signature format is open.

5 Parameters, Implementation and Applications

5.1 Parameters and sizes

Table 1 lists the three parameter sets. Pre-signatures and signatures have the same encoding and length, and an adapted signature is an ML-DSA signature. A statement $\mathbf{R} \in R_q^k$ is sent uncompressed and takes $k \cdot 256 \cdot 23$ bits. These lengths do not include the context string, a proof of knowledge for the statement, or protocol messages.

Pre-signing needs more iterations than signing for two reasons: the response bound is tightened by η, which rejects an additional fraction $\varepsilon_{\mathbf{z}}$ of iterations, and condition (C1) rejects a fraction of about ε_{C} (Section 4.7); condition (C2) concerns at most ω coefficients and has a negligible effect. The relative increase in the expected number of iterations is therefore about $1/((1-\varepsilon_{\mathbf{z}})(1-\varepsilon_{\mathsf{C}})) - 1$. The same two quantities are the trigger rates of the tests of Proposition 2. $\mathsf{PreVerify}$ costs about one ML-DSA verification plus a vector addition and the check of C, Adapt is one $\mathsf{PreVerify}$ plus a vector addition, and $\mathsf{Extract}$ is one matrix–vector product.

5.2 Implementation and experiments

Our prototype implements ML-DSA-44 in Rust. It builds on the crate `fips204` 0.4.6: the adaptor module calls the crate's ring arithmetic, encodings, message framing, SHAKE and sampling functions, and leaves its key generation, signing and verification code untouched. Adapted and filtered signatures were also checked with the verifiers of RustCrypto `ml-dsa` 0.1.1 and of PQClean, called through `pqcrypto-mldsa` 0.1.2 [Impl]. These are three implementations of verification; pre-signing exists in one implementation only.

Each trial uses a fresh key, a fresh statement and a 32-byte message, with randomness from ChaCha20 under a recorded seed. Half of the correctness trials use a nonempty context, and a quarter of the pre-signatures use the deterministic mode. We report two runs, with seeds 20261002 and 777, each consisting of 100 000 correctness trials, 200 000 trials without C, 50 000 successful filtered adaptations and 10 000 pairs for the mask test. Reaching an iteration cap is reported as an error; neither run reached one, and the largest number of pre-signing iterations observed was 44 and 61. Table 3 combines the two runs. Two further diagnostic runs are reported in the text: the naive-mask test of Remark 2 (seed 20261008) and the public boundary test of Section 4.7 (seed 20261019); the last row of Table 3 is from the former.

Table 1: Parameters, sizes in bytes, and predicted overheads. The LAS figure is from [EEE20] and refers to different parameters.

	ML-DSA-44	ML-DSA-65	ML-DSA-87
(k, ℓ)	$(4, 4)$	$(6, 5)$	$(8, 7)$
η; τ	2; 39	4; 49	2; 60
γ_1	2^{17}	2^{19}	2^{19}
γ_2	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
ω	80	55	75
signature / pre-signature	2420	3309	4627
statement $\mathbf{R}$	2944	4416	5888
verification key	1312	1952	2592
extra PreSign iterations (predicted)	4.3%	3.7%	2.7%
$\varepsilon_{\mathbf{z}}$; ε_{C}	1.55%; 2.65%	0.97%; 2.60%	0.68%; 1.94%
LAS [EEE20], signature		≈ 3210	

Table 2: Published lattice-attack estimates for the ML-DSA instances related to our assumptions: BKZ block size and classical core-SVP exponent from [BDK+21, Table 1], whose parameters coincide with those of ML-DSA. The SelfTargetMSIS and MSIS rows repeat the SIS estimates of [BDK+21] (for MSIS, those given for strong unforgeability). They are not a separate concrete assessment of the digest form of SelfTargetMSIS used in Theorem 2, and they do not account for the conversion loss of Remark 5.

Instance	Used for	ML-DSA-44	ML-DSA-65	ML-DSA-87
$\mathsf{MLWE}_{k,\ell,\eta}$	keys, statements	423 / 123	624 / 182	863 / 252
$\mathsf{SelfTargetMSIS}_{k,\ell+1,\zeta}$ (SIS est.)	extractability	423 / 123	638 / 186	909 / 265
$\mathsf{MSIS}_{k,\ell,\zeta'}$	unique extractability	417 / 121	602 / 176	868 / 253

The average number of iterations was 4.364 for signing and 4.569 for pre-signing, an increase of 4.7% (predicted 4.3%). The signing figure agrees with the value 4.36 that Kwiatkowski obtains by accounting for all rejection conditions [Kwi26], now also listed in NIST's errata [NIST26]. Sign' needed 10.8% more iterations than signing. With the statement in the mask seed, the smallest $\|\widehat{\mathbf{z}} - \widehat{\mathbf{z}}'\|_\infty$ over the 20 000 deterministic pairs with distinct statements was 235 423, far from the value $2\beta = 156$ produced by mask reuse, and repeated runs on the same message and statement reproduced the pre-signature byte for byte. None of the three tests of Section 4.7 separated filtered adapted signatures from signatures of Sign'.

The test suite covers encoding round trips and rejection of non-canonical statement coefficients, ring multiplication against a reference, byte equality between the generic signing loop and the crate's signing function, the full protocol, mask separation for distinct statements, rejection on a digest mismatch, and rejection of invalid encodings. Python scripts recover the witness of the noiseless relation, check Lemma 1 exhaustively, and compute the bound of Proposition 1. Source code, logs and scripts are available at `https://codeberg.org/lis111/mldsa-adaptor`.

5.3 Application: atomic swaps

In a cross-chain atomic swap, Alice generates a statement and a witness, each party pre-signs the other's transaction with respect to the statement, Alice adapts Bob's pre-signature to claim Bob's coins, and Bob extracts the witness from the chain and adapts Alice's pre-signature.

The knowledge gap matters here: the witness Bob extracts is only guaranteed to lie in $\mathcal{R}'$, and with a large $\mathbf{r}_1$ it cannot be used for adaptation. As in LAS [EEE20], Alice attaches a zero-knowledge argument of knowledge of a witness in $\mathcal{R}_\eta$ at the start of the swap. Soundness alone does not suffice, because the argument below needs the witness itself, and the extractor of the argument must be usable in the protocol's environment. Bob issues a single pre-signature $\widehat{\sigma}$ on his transaction and signs nothing else on it. If Alice's posted signature σ does not extract from $\widehat{\sigma}$, it is a forgery in the sense of Theorem 2. Otherwise let w be the

Table 3: Combined results of the two Rust runs (ML-DSA-44); the last row is from a separate diagnostic run (Remark 2).

Event	Count	Rate
correctness: pre-verification, three verifiers and exact extraction all succeed	200000/200000	100%
without C: adapted signature rejected	2564/400000	0.641%
fresh signature, D_1 fires	3168/200000	1.584%
fresh signature, D_2 fires	5190/200000	2.595%
fresh signature, $D_1 \vee D_2$ fires	8266/200000	4.133%
adapted signature, D_1 or D_2 fires	0/200000	0%
adapted signature, D_3 (needs $\mathbf{t}_0$) fires	1272/200000	0.636%
$\mathsf{FilterAdapt}$ rejects	6096/106096	5.746%
filtered signatures accepted by three verifiers	100000/100000	100%
naive mask derivation: short difference $\widehat{\mathbf{z}} - \widehat{\mathbf{z}}'$	4980/20000	24.900%

tight witness given by the knowledge extractor and $\sigma_0 = \mathsf{Adapt}(vk, M, \mathbf{R}, \widehat{\sigma}, w)$, which is valid and extracts to w (Theorem 1). If $\sigma \neq \sigma_0$, the tuple $(M, \mathbf{R}, \widehat{\sigma}, \sigma, \sigma_0)$ breaks unique extractability (Theorem 4). Hence $\sigma = \sigma_0$, and Bob extracts exactly $w \in \mathcal{R}_\eta$. This avoids taking the difference of a wide and a tight witness, which would need MSIS with bound $\zeta' + \eta$. With several pre-signatures on the transaction, extractability only guarantees extraction from one of them, and the protocol would have to identify which.

Since each relation uses the signer's own matrix, the two chains verify under different matrices $\mathbf{A}_A$ and $\mathbf{A}_B$. Alice chooses one $\mathbf{r}_1$ and two independent errors and publishes

$$\mathbf{R}_A = \mathbf{A}_A\mathbf{r}_1 + \mathbf{r}_{2,A}, \qquad \mathbf{R}_B = \mathbf{A}_B\mathbf{r}_1 + \mathbf{r}_{2,B},$$

together with an argument of knowledge for both equations with all components bounded by η; this costs about one argument of twice the dimension. Bob pre-signs with respect to $\mathbf{R}_B$; after Alice has adapted, Bob extracts $(\mathbf{r}_1, \mathbf{r}_{2,B})$, computes $\mathbf{r}_{2,A} = \mathbf{R}_A - \mathbf{A}_A\mathbf{r}_1$, and adapts Alice's pre-signature. The argument of the previous paragraph, applied to the pair of equations, shows that the extracted $\mathbf{r}_1$ is Alice's. Both chains must use the same parameter set, so that $\mathbf{r}_1$ has the same dimension and bound on both sides. The pseudorandomness of $(\mathbf{R}_A, \mathbf{R}_B)$ is an MLWE instance with $k_A + k_B$ rows and a shared secret, which differs from the instance protecting a single key. We have not estimated its concrete hardness separately. With a shared matrix (Remark 1) a single statement serves both chains.

This is an argument at the level of signatures. A complete protocol also has to handle the composition of the argument of knowledge with the protocol, binding of transactions, timeouts and refunds, finality and concurrent sessions, which we do not treat. Chains that already verify ML-DSA signatures need no new verification logic. The same properties are relevant for conditional payments, where a service provider faces many clients under one key and may receive several pre-signing requests on the same message: extractability with several pre-signatures, unique extractability, and the mask derivation of Remark 2 are what make this safe.

5.4 Assumptions

Table 4 lists the model and assumptions behind each result. The instances are related but not identical.

6 Limitations

We do not claim unlinkability in the sense of [GSST24]: the advantage of the witness-holder's test equals the probability that it fires on a fresh signature, for which we have a heuristic estimate and a measurement of about 4% for ML-DSA-44 but no proven lower bound, and we give no upper bound on the advantage of an observer without the witness (Section 4.7); the filter of Proposition 3 does not repair this and gives up adaptability. Applications such as atomic swaps need an argument of knowledge for $\mathcal{R}_\eta$, as LAS does; its cost is not included in Table 1, and with current proof systems the argument, not the pre-signature, dominates communication. All proofs are in the classical random oracle model, with ideal masks and capped signing;

Table 4: Model and assumptions behind each result.

Result	Model, assumptions and additional terms
Correctness, adaptability (Lemma 2, Thm. 1)	none; canonical encodings, witness in $\mathcal{R}_\eta$
Extractability (Thm. 2)	uniform $\mathbf{A}$, classical ROM, ideal masks, caps; Δ_{cap}, $\mathsf{MLWE}_{k,\ell,\eta}$, $\mathsf{SelfTargetMSIS}_{G,k,\ell+1,\zeta}$, ε_μ
Actual mask derivation	in addition Δ_{mask}
Uncapped signing (Cor. 1)	in addition $\delta_{\mathsf{S}}(K_{\mathsf{S}})$
Adaptor unforgeability (Thm. 3)	extractability, $\mathsf{MLWE}_{k,\ell,\eta}$ for statements, $\mathsf{MSIS}_{k,\ell+1,\zeta''}$
Unique extractability (Thm. 4)	$\mathsf{MSIS}_{k,\ell,\zeta'}$, collision resistance of H_{c}
Atomic swap (Sec. 5.3)	the above, an argument of knowledge for $\mathcal{R}_\eta$, one pre-signature per transaction; stacked MLWE with two matrices
Filter (Prop. 3)	fixed inputs, ideal transcripts, fresh masks, success conditioning; not an unlinkability result

the mask term Δ_{mask} and the tail term δ_{S} for uncapped signing are not quantified, and a treatment in the quantum random oracle model along the lines of [KLS18] is open. Extractability uses the digest form of SelfTargetMSIS, and the published estimates in Table 2 assess neither this form, nor the instance $\mathsf{MSIS}_{k,\ell+1,\zeta''}$ of Theorem 3, nor the stacked MLWE instance of Section 5.3 separately (Remark 5). The prototype covers ML-DSA-44 only and has not been reviewed for constant-time behaviour or other side channels.

7 Conclusion

We gave an adaptor signature whose keys, pre-signature format and adapted signatures are those of ML-DSA, so adapted signatures pass the unmodified FIPS 204 verification. A public condition on the recomputed commitment makes every pre-verified pre-signature adaptable with any witness of the tight relation, and extractability, unique extractability and adaptor unforgeability reduce without rewinding to MLWE, to a digest form of the SelfTargetMSIS problem behind the unforgeability of ML-DSA, and to the MSIS instance behind its strong unforgeability, with one extra column for adaptor unforgeability. The bounds add an oracle-simulation term that is small for capped signing and pre-signing, and the MLWE term of adaptor unforgeability carries a factor $2Q_{\mathsf{R}}$. Unlinkability is not among these results: in our experiment, tests that need the witness fire on a few percent of fresh signatures and a public test on about 0.5%. Unlinkability together with adaptability in the ML-DSA signature format remains open. LAS+ [WYX+26] obtains a weak form for the uncompressed variant of LAS by changing its signing algorithm; we have kept the signing algorithm of FIPS 204 unchanged, which rules out that route here.

A The oracle-simulation bound

We prove Proposition 1. The proof follows the hybrids of Barbosa et al. [BBD+23] for Fiat–Shamir with aborts and keeps track of what their theorem does not state explicitly in our setting: the adversary queries a family of schemes $\mathsf{ID}_{\mathbf{R}}$ (with $\mathbf{R} = \mathbf{0}$ and the untightened bound for signing), with statements chosen adaptively, all sharing the oracle H_{c}; and every query stops after a fixed number of iterations.

Setting. Fix the matrix and the key, and write $\varepsilon = \varepsilon(\mathbf{A})$ for the guessing bound (4); the average over $\mathbf{A}$ is taken at the end. Before each signing-type query the adversary fixes the message and, for pre-signing, the statement. The mask of each iteration is sampled after this history, so (4) applies to every history and to every target set fixed before the mask is drawn: a fresh commitment hits a set T of points with probability at most $|T|\,\varepsilon$. One iteration samples $\mathbf{y}$, forms the point $P = (\mu, \mathsf{w1Encode}(\mathbf{w}_1))$ with $\mathbf{w}_1 = \mathsf{HighBits}(\mathbf{A}\mathbf{y} + \mathbf{R})$, obtains a digest for P, and accepts or rejects. All counts include the verification of the adversary's final output.

Hybrids.

- Real: each iteration queries $H_{\mathsf{c}}(P)$, using the existing value if P is already defined, and then accepts or rejects.
- Prog: each iteration samples a digest locally and sets $H_{\mathsf{c}}(P)$ to it, overwriting any existing value, whether the iteration is then accepted or rejected.
- Trans: as Prog, except that rejected iterations leave H_{c} untouched; only an accepted iteration programs its point. If the cap is reached, the query returns $\perp$.
- Sim: the simulator of Section 4.2, with the box coin and no short secret; it programs on success and returns $\perp$ at the cap.

Every hybrid uses the same caps. Iteration counts of different queries may be correlated through the adversary's adaptive choices; the proof only uses $N \le L$ from Lemma 4(iii).

From Real to Prog. The two hybrids proceed identically until an iteration forms a point that is already defined, by an adversary query or by an earlier iteration; call this event B_1. At the start of the j-th iteration at most $Q_{\mathsf{H}} + j - 1$ points are defined, and the fresh commitment hits one of them with probability at most $(Q_{\mathsf{H}} + j - 1)\varepsilon$, whatever happened before. Summing over at most L iterations (counting iterations that do not occur only increases the sum),

$$\Pr[B_1] \le \varepsilon \sum_{j=1}^{L} (Q_{\mathsf{H}} + j - 1) = \varepsilon\Big(Q_{\mathsf{H}} L + \frac{L(L-1)}{2}\Big).$$

Until the first such event, the answers of H_{c} and the local digests can be coupled iteration by iteration.

From Prog to Trans. The hybrids differ only at the points of rejected iterations. In Prog such a point holds the digest that caused the rejection, a value correlated with the rejection; in Trans it is undefined and would be answered freshly. This is the bias identified in [BBD+23]. A query to such a point made before the iteration is covered by B_1. A later iteration that lands on it samples a fresh local digest in both hybrids; if that iteration is accepted, both program the point, and if it is rejected, the difference persists and is covered by the event below. What remains is that the adversary, after the query, asks H_{c} at the point of a rejected iteration, or the final verification does; call this B_2. We bound B_2 in Trans, one query at a time.

Condition on the full history H_i before the i-th signing-type query, which fixes its message, its statement and its cap K_i. In Trans its iterations are independent given H_i, so we may think of them as sampled in advance, $D_1, \dots, D_{K_i}$. Let E_t be the event that $D_1, \dots, D_t$ are all rejected, and define the bounded suffix output

$$D^{(t)} = \begin{cases} \text{the first accepted transcript among } D_{t+1}, \dots, D_{K_i}, & \text{if there is one,} \\ \perp, & \text{otherwise.} \end{cases} \tag{8}$$

In particular $D^{(K_i)} = \perp$, and when the suffix output is $\perp$ no point is programmed. On E_t the query returns $D^{(t)}$, and the rejected iterations have not changed H_{c}.

Fix a later query x of the adversary (or the final verification query). Let $\tilde{x}$ be the query that the adversary would make in its place if the i-th query returned $D^{(t)}$ and the interaction continued with independent future randomness; if there is no such query, let $\tilde{x}$ be a value that matches no point. Given H_i and E_{t-1}, $\tilde{x}$ is independent of D_t, and on E_t we have $x = \tilde{x}$. Writing $P(D_t)$ for the point of D_t, including its message part,

$$\begin{aligned} \Pr[E_t \wedge x = P(D_t) \mid H_i] &\le \Pr[E_{t-1} \wedge \tilde{x} = P(D_t) \mid H_i] \\ &= \Pr[E_{t-1} \mid H_i] \cdot \Pr[\tilde{x} = P(D_t) \mid H_i, E_{t-1}] \le \Pr[E_{t-1} \mid H_i]\, \varepsilon. \end{aligned}$$

The step that matters is dropping the rejection of D_t before applying the guessing bound, so that the bound is applied to an iteration that is not conditioned on being rejected. The suffix has to allow $\perp$: if it were

defined as the first accepted transcript after the cap, executions that end in $\perp$ could not be coupled with it. Summing over t,

$$\sum_{t=1}^{K_i} \Pr[E_{t-1} \mid H_i] = \mathbb{E}[N_i \mid H_i] \le K_i,$$

where N_i is the number of iterations of the i-th query. Summing over the at most $Q_{\mathsf{H}} + 1$ later queries and over all signing-type queries,

$$\Pr[B_2] \le (Q_{\mathsf{H}} + 1)\, L\, \varepsilon.$$

New statements chosen later enter only through the future interaction that defines $\tilde{x}$, so they do not affect its independence from D_t; the guessing bound holds for every statement.

From Trans to Sim, and the average over A. By Lemma 4(i), the translation $\mathbf{y} \mapsto \mathbf{y} + c\mathbf{s}_1$ matches accepted iterations, and the box coin matches the probability that an iteration passes the box check. Run with the same caps, the two hybrids produce identically distributed outputs, programmed points and $\perp$, so this step costs nothing.

Adding the first two steps and rounding up, for a fixed matrix

$$\Delta(\mathbf{A}) \le \big[2(Q_{\mathsf{H}} + 1)L + L^2\big]\varepsilon(\mathbf{A}).$$

Averaging over $\mathbf{A}$ on the event $\{\mathrm{cr}(\mathbf{A}) \le D\}$, and bounding the distance by 1 on its complement, gives (6). We do not discard the event that some component is singular: for square matrices it has probability about 2^{-15}, too large to be charged as a security loss. The family enters only through the uniformity of (4) in $\mathbf{R}$, through the caps, and through the conditioning on the history in the two counting arguments. With actual masks the same proof applies after the masks are replaced by ideal ones at cost Δ_{mask}, repeated deterministic queries being answered by replay.

B Corank distribution and numerical values

For a uniform $\ell \times \ell$ matrix over $\mathbb{F}_q$, the corank equals a with probability

$$p_a = q^{-a^2} \prod_{i=0}^{\ell-a-1} \frac{(1 - q^{i-\ell})^2}{1 - q^{i-(\ell-a)}}, \qquad 0 \le a \le \ell,$$

with the empty product equal to 1. Let $g(X) = \sum_{a=0}^{\ell} p_a X^a$. The 256 NTT components of a uniform matrix over R_q are independent, so $\mathrm{cr}(\mathbf{A})$ is distributed according to the coefficients $p_s^{(256)}$ of $g(X)^{256}$, and

$$\varepsilon_D = \Big(\frac{\alpha + 1}{2\gamma_1}\Big)^{256\ell} \sum_{s=0}^{D} q^s p_s^{(256)}, \qquad \delta_D = \sum_{s>D} p_s^{(256)}.$$

Our script computes the single-component probabilities exactly as rationals, rounds them up to 450 decimal digits, and performs the convolution and all sums with upward rounding. The tail δ_D is summed directly rather than computed as $1 - \mathrm{CDF}$, and no high-corank terms are dropped. The comparison with the stated binary bounds is made directly; logarithms in Table 5 are for display.

Table 5: Simulation bound with at most 2^{63} signing and 2^{63} pre-signing queries and caps $K_{\mathsf{S}} = K_{\mathsf{P}} = 4096$.

Parameter set	$\log_2 Q_{\mathsf{H}}$	D	$\log_2 \varepsilon_D$	$\log_2 \delta_D$	bound
ML-DSA-44	128	11	−409.393	−209.196	$\le 2^{-204}$
ML-DSA-65	192	44	−1115.926	−867.157	$\le 2^{-846}$
ML-DSA-87	256	63	−1591.987	−1268.343	$\le 2^{-1258}$

These numbers bound the simulation term only. The MLWE, SelfTargetMSIS, MSIS and collision terms are separate, as are Δ_{mask} and, for uncapped signing, δ_{S}.

References

[AEE+21] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, S. Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. ASIACRYPT 2021. Cryptology ePrint Archive 2020/476.

[BBD+23] M. Barbosa, G. Barthe, C. Doczkal, J. Don, S. Fehr, B. Grégoire, Y.-H. Huang, A. Hülsing, Y. Lee, X. Wu. Fixing and mechanizing the security proof of Fiat-Shamir with aborts and Dilithium. CRYPTO 2023. Cryptology ePrint Archive 2023/246.

[BDK+21] S. Bai, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé. CRYSTALS-Dilithium: Algorithm specifications and supporting documentation (version 3.1). 2021.

[BP18] L. Groot Bruinderink, P. Pessl. Differential fault attacks on deterministic lattice signatures. IACR TCHES 2018(3). Cryptology ePrint Archive 2018/355.

[DKL+18] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, D. Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. IACR TCHES 2018(1). Cryptology ePrint Archive 2017/633.

[DOY22] W. Dai, T. Okamoto, G. Yamamoto. Stronger security and generic constructions for adaptor signatures. INDOCRYPT 2022. Cryptology ePrint Archive 2022/1687.

[DTS+26] N. Döttling, S. A. K. Thyagarajan, P. Soni, J. Taylor, H. Waldner, R. Zanotto. A general approach to adaptor signatures. Cryptology ePrint Archive 2026/1889.

[EEE20] M. F. Esgin, O. Ersoy, Z. Erkin. Post-quantum adaptor signatures and payment channel networks. ESORICS 2020. Cryptology ePrint Archive 2020/845.

[GSST24] P. Gerhart, D. Schröder, P. Soni, S. A. K. Thyagarajan. Foundations of adaptor signatures. EUROCRYPT 2024. Cryptology ePrint Archive 2024/1809.

[Impl] Implementations used in the experiments: `fips204` 0.4.6; RustCrypto `ml-dsa` 0.1.1; PQClean ML-DSA via `pqcrypto-mldsa` 0.1.2. Exact dependency versions are fixed by the `Cargo.lock` of our implementation.

[KLS18] E. Kiltz, V. Lyubashevsky, C. Schaffner. A concrete treatment of Fiat-Shamir signatures in the quantum random-oracle model. EUROCRYPT 2018.

[Kwi26] K. Kwiatkowski. On the expected number of iterations in ML-DSA signing: A complete accounting of the rejection conditions. Cryptology ePrint Archive 2026/2456.

[Lyu09] V. Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. ASIACRYPT 2009.

[MA26] A. Mohammadali, R. AlTawy. L-BAS: A lattice-based blind adaptor signature scheme. Cryptology ePrint Archive 2026/1752.

[NIST24] NIST. FIPS 204: Module-Lattice-Based Digital Signature Standard. 2024.

[NIST26] NIST. FIPS 204: Potential updates (errata), entry of 31 July 2026 on Section 4 and Appendix C. `https://csrc.nist.gov/files/pubs/fips/204/final/docs/fips-204-potential-updates.xlsx`.

[Poe17] A. Poelstra. Scriptless scripts. 2017.

[PS26] A. Pellet-Mary, M. Seck. On the properties of HighBits and LowBits functions and their applications. AFRICACRYPT 2026. Cryptology ePrint Archive 2026/895.

[QPM+23] X. Qin, S. Pan, A. Mirzaei, Z. Sui, O. Ersoy, A. Sakzad, M. F. Esgin, J. K. Liu, J. Yu, T. H. Yuen. BlindHub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts. IEEE S&P 2023. Cryptology ePrint Archive 2022/1735.

[TMM21] E. Tairi, P. Moreno-Sanchez, M. Maffei. Post-quantum adaptor signature for privacy-preserving off-chain payments. FC 2021. Cryptology ePrint Archive 2020/1345.

[WYX+26] J. Wang, G. Yang, H. Xue, M. Hao, R. H. Deng. LAS+: Lattice-based adaptor signature with stronger security. ISC 2026, Rennes, Springer LNCS, to appear. Preprint: `https://isc2026.github.io/papers/paper_4244.pdf`.

[Thesis26] WangQi. Research on Lattice-Based Adapter Signature Scheme. Master's thesis, Institute of Information Engineering, Chinese Academy of Sciences, 2026. In Chinese.